

claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf_5de56b64-647\agent-a78643e55a1f9b522.jsonl`  
- SHA-256: `106b10c249b105e7d77db97b7a13db19948ba8ea84058b0e023b590ae3ff1c48`  
- Source modified: `2026-06-21T14:41:18+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_5de56b64-647`  
- session_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`
```

Transcript

1. user (2026-06-21T14:36:44.491Z)

Repo: C:\Users\avidu\Projects\badminton-highlight-indexer. Scope the M11 data-flywheel work item precisely (it is PRIVACY-SENSITIVE ? consent/data).

Read docs/COMPUTE_DECOUPLED_SERVING/** (esp. the tracker / README for M11), backend/utils/workspace.py (for_video), backend/eval/experiment.py (compare), backend/eval/promotion.py (promote_if_served_safe), and grep for any existing flywheel/consent code.

Determine:

- Exactly what files M11 creates/touches (the plan says backend/flywheel.py: capture->store->shadow-eval->goldenize, behind flywheel.* config = OFF + a faked consent_attested).
- What already exists to reuse vs what's net-new. Cite file:line for workspace.for_video, experiment.compare, promotion.promote_if_served_safe signatures.
- The privacy gating surface: what consent fields/flags are needed, what must be default-OFF, what is owner/counsel-gated (real consent text/DPA) vs claude-doable plumbing.
- Test surface: what can be tested offline/mockd here.
- Does M11 need a DB migration? The plan says M9-consent cols = a new v7 migration (M8 took v6). Confirm the current PRAGMA user_version in backend/infrastructure/database.py and what version M11/M12 would take. Cite file:line.

Give a realistic effort + the cleanest isolated-PR shape.

2. assistant (2026-06-21T14:36:46.633Z)

I'll help you scope the M11 data-flywheel work systematically. Let me start by exploring the repository structure and reading the key files.

3. user (2026-06-21T14:36:47.953Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/01-EXISTING-PLANS-AND-GAP.md  
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md  
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/03-PLATFORM-OPTIONS.md  
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md  
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md  
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/06-RISKS-AND-OPEN-QUESTIONS.md  
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md  
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md  
C:\Users\avidu\Projects\badminton-highlight-
```

```
indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/HANDOFF.md
C:\Users\avidu\Projects\badminton-highlight-
indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/README.md
C:\Users\avidu\Projects\badminton-highlight-
indexer/docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md
C:\Users\avidu\Projects\badminton-highlight-
indexer/docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_truly-local_TEMPLATE.md
C:\Users\avidu\Projects\badminton-highlight-
indexer/docs/COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md
```

4. assistant (2026-06-21T14:36:49.930Z)

Let me check the correct directory path and read the M11 tracker/README:

5. user (2026-06-21T14:36:51.360Z)

```
1      # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3      > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** .
**Owner:** Avi . **Created:** 2026-06-21 . **Last updated:** 2026-06-21
4      > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to
`docs/archives/past_projects/` when DONE)
5      > **Tracking anchors:** $7 progress tracker is the source of truth; pointer in memory
`current-state` + `docs/NEXT_STEPS.md` once work starts.
6      > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) $3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7      > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
**compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an
afterthought.
8      > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10     ---
11
12     > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user
*confirms* (D15).
13
14     ## 0. TL;DR
15     The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three
deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch
on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process
vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB
/ mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local**
(local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive /
bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus
**cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the
same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the
**parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.
16
17     ## 1. Problem & goal
18     - **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4)
and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero +
cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local
mode, with the **core (detect+stitch) unaffected** by which one runs.
19     - **The two user-facing outcomes** (owner, 2026-06-21):
20     - ***(a) Hosted service** ? operate on small **local uploads (`<2GB`)** *and* load from
**gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching**
(stitching is compute-intensive ? run it where its cost is accounted for).
```

21 - ****(b) Truly-local**** ? on a local machine with videos on ****local drives/USB + a local GPU****, run inference ****and**** stitching ****completely locally****.

22 - *****"Good" looks like:**** the same `video ? rallies ? reel` core gives identical windows + stitched ranges whether it runs on a laptop or Cloud Run; switching is a ****config/startup choice****, never a code branch in the core.

23

24 **## 2. Decisions locked**

25

26 | # | Decision | Source | Implication |

27 |---|-----|-----|-----|

28 | D1 | The core is a ****pure 3-stage function**** (DETECT/WINDOW/STITCH); ****no profile branch inside it****. | design `[verified]` | All profile difference lives in config + the two registries. |

29 | D2 | ****Two registries:**** `ComputeBackend` (where) selected by `deployment.profile` + `compute\_target`; `StorageBackend` (what-bytes) by `input\_backend`/`storage.backend`. | ADR-014 | Realizes PLATFORM_ARCHITECTURE's `local`/`hosted`/`byo\_worker`. |

30 | D3 | ****Profiles shipped:**** `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo\_worker` reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |

31 | D4 | In ****cloud-serving, STITCH runs on Cloud Run**** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would hide real cost. |

32 | D5 | ****Parity enforced at WINDOW+STITCH**** (byte-exact for a fixed trajectory); ****detect**** is byte-exact only same-GPU. | deploy report 2026-06-20 | Cross-GPU asserted at the rally/window level, never CSV bytes. |

33 | D6 | ****Default-OFF, smallest-safe-first, one PR per milestone****; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report. | [[launch-report-convention]] | Zero-regression discipline. |

34 | D7 | ****Cloud Run cold-start: SCALE-TO-ZERO**** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). ****No warm pool**** (that ~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane only on a UX complaint. |

35 | D8 | ****Pricebook = a versioned DB table****; cost computed in ****USD**** (matches GCP billing = one source of truth), ****displayed in INR**** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |

36 | D9 | ****Edge auth = Cloudflare Access**** (reuse the site's existing CF). Lock the Cloud Run ****ingress to the CF proxy**** + ****verify the `Cf-Access` JWT signature**** (so a direct hit to the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |

37 | D10 | ****Free-tier "data-for-reels" trade:**** free reels in exchange for ****training rights**** (uploaded footage + derived trajectories/labels); ****paid tier = no-train**** (privacy is the paid feature). ****Carve-outs:**** train ONLY on attested ****ADULT**** footage (minor footage ? reel yes, training pool ****NO**** ? DPDP/GDPR need verifiable parental consent); train on ****DERIVED**** data + ****auto-delete raw****; ****ToS counsel-reviewed****; live training-on-uploads ****gated to M9**** (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel. |

38 | D11 | ****Quality floor: Gemini handoff ON**** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a ****Launch Report****; the owned model runs in ****shadow/eval**** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |

39 | D12 | ****Data-flywheel harness is IN SCOPE (owner add, Q5):**** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? ****reliably store**** in the per-video workspace/bucket ? ****run shadow evals**** (owned-vs-Gemini, dual-eval-set) ? ****promote good ones to the golden TRAINING set**** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new ****M11****; ties D10 (consent) + `data\_pipeline/` + the eval/experiment harness. |

40 | D13 | ****Profile is user-switchable ANYTIME ? a headline FEATURE.**** The same user runs ****local today, cloud tomorrow, back to local**** ? it's just a config/startup choice; the pure core gives ****equivalent**** output either way. ****Advertise it**** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the ****rally/window**** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is ****between* jobs.** |

41 | D14 | ****NORTHSTAR ? operate toward this:**** a ****mobile, app-like**** flow ? point at a ****Google Drive**** video ? ****fully cloud-native**** run ? the stitched, ****ready-to-share**** reel lands ****back in Drive, next to the source****. | owner 2026-06-21 | Implies a ****`gdrive-api` (OAuth) StorageBackend**** (read source + write reel back) ? distinct from the local ****mount****

backend; ****resolves S8 #7****, new ****M12****. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it). |

42 | D15 | ****Auto-detect SUGGESTS**, the user **CONFIRMS** ? execution is **NEVER** a surprise.****** A GPU owner may still choose cloud; ****cloud (= spend)** is never entered silently******; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends S4.3 (auto-detect = a proposed default, not an auto-decision). |

43 | D16 | ****M6/M7 implementation SHAPE** + the cheap S8 picks (owner 2026-06-21):****** (a) ****Cloud Run JOBS**** (not Services-with-GPU) for the detect+stitch unit, ****bucket-poll** for completion****** (reuses `JobWaiting`/`claim\_due\_job` verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) ****stitch ENCODE = libx264**** (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? ****NVDEC** for **DECODE** only****** (a speed lever that never touches output bytes); ****NVENC** encode is **DEFERRED** behind a config knob******, revisit only if M7 cost data justifies relaxing parity (S8 Q8); (c) ****input cap = hard-reject`<2GB`**** for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? S8 Q9). | owner 2026-06-21 | M6/M7. Resolves S8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed). |

44 | D17 | ****Real cloud verification BUDGET = \$100 / ?10k cap** (owner 2026-06-21):****** the owner authorized actual GCP spend for due-diligence + production-grade testing/verification of M5?M7 (real L4 run, the M5 runlog, the M7 DEPLOY_REPORT), ****provided the cap is respected**** ? confirm exact cmd + hourly cost before each spend, prefer the smallest viable run + cheapest-adequate GPU, and ****DELETE** every resource after****** ([[gcp-vm-always-delete]]). Removes the M6/M7 "spend" gate within the envelope; the calibrated pricebook (M8) + counsel ToS (M9) + OAuth app (M12) stay owner/counsel-side. | owner 2026-06-21 | Unblocks the real M5/M7 runs (not just the code) within the cap. |

45

46 ## 3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*

47 This project is the latest step in a long decision chain; each ADR contributed a piece and a ****subtlety that carries forward****:

48

49 | ADR / doc | Contributed | Subtlety carried into this project |

50 |---|---|---|

51 | ****ADR-005**** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the ****container image**** (ffmpeg, fonts, weights) stay commercial-clean. |

52 | ****ADR-008**** | Owned-model topology; ****train==serve** featurizer single-sourcing****** | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to ****local/cloud****. |

53 | ****ADR-009**** | KhelSutra Guru; ****local-first delivery**** | The ****truly-local profile**** is the direct continuation ? privacy/offline self-host stays first-class. |

54 | ****ADR-010**** | DECOUPLED: DO?GCP pivot | The cloud-compute origin (GPU + co-located bucket). |

55 | ****ADR-011**** | DECOUPLED scope = M0?M2+M3.5; ****deferred M3 (serving) + M4** (billing/cost-guard)******; overrode the "rent nothing / not a service" posture | ****This project realizes M3/M4.**** Carries S06's owner-gated gates: ****DPA/consent** for non-owner uploads, collector `md5` keying, WASB-C3******. |

56 | ****ADR-012**** | GPU-native; TPU deferred; ****Cloud Run+GPU scale-to-zero = serving model****; ****NVDEC**** named as the decode lever | The serving substrate + the 7-step seed scope this expands. |

57 | ****ADR-013**** | Drop D0; ****GCP sole compute stack****; \$200 D0 credits ? non-compute only | The provider for cloud-serving; D0 Spaces remains only a possible S3 *storage* target. |

58 | ****? ADR-014 (new)**** | ****Compute-decoupled serving:**** one pure core, profile-selected compute+storage; stitch-where-metered | This doc. ****Refines**** ADR-011/012, does ****not**** supersede them. |

59 | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The abstraction this realizes in code. |

60 | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop` backend | The truly-local profile = these, productionized. |

61 | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB hardcodes `device='cuda'` with ****no CPU fallback**** ? a portability gap M4 closes. |

62 | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; ****cross-GPU sub-pixel parity finding**** | Why D5 (parity at window level, not CSV bytes). |

63

64 ## 4. Design / architecture

65 See architecture.svg for the picture. ****The core never changes; the box around it does.****

```

66
67     ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
68     1. DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ?`
69     CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable
70     impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ?
71     emit the identical CSV schema**. The trajectory CSV is the stable artifact boundary.
72     2. WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?)`
73     ? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same
74     trajectory + config = byte-identical candidate windows. Used by every runner and the
75     eval/regression stack verbatim.
76     3. STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments,`
77     out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffmpeg ? watermark
78     ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
79
80     Non-goal (the trap to avoid):** no code branch inside detect/window/stitch keyed on
81     profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-
82     identical to CONTROL).
83
84     ### 4.2 The deployment profiles
85     | Profile | Compute | Storage | Orchestration | When |
86     |---|---|---|---|---|
87     | truly-local | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto`
88     (WSL); stitch in-process | `local` (or `gdrive` = mounted Drive) | `python -m backend.main`
89     or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-
90     box ? owner (b) |
91     | cloud-serving | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda);
92     detect AND stitch on the same Cloud Run job | `gcs` (or gdrive mount / assembled upload);
93     per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job
94     pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted
95     SaaS ? owner (a) |
96     | cpu-mock | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) |
97     `local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free
98     dev |
99     | byo_worker (deferred) | tenant GPU via outbound pull agent | tenant's own bucket
100    (signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ?
101    DEFERRED, enum/seam reserved |
102
103    ### 4.3 Profile selection (the "startup/setup config") `[design]`
104    ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-
105    detect in `load_config`:
106    - `deployment.profile` ? {truly-local | cloud-serving | cpu-mock} (empty = today's
107    manual knobs, fully backward-compatible).
108    - Selecting a profile applies a coherent preset bundle of existing knobs that
109    today drift independently: INPUT (new `input_backend`/`input_root`, parallel to the output-
110    side `StorageConfig`), COMPUTE (new `compute_target` enum locking `detector_impl` +
111    `skip_ai_handoff` + workspace strategy), STORAGE
112    (`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
113    - Precedence: built-in defaults ? profile preset ? `config.json` ? env
114    (`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob
115    stays individually overridable.
116    - Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution):** the env
117    gives a proposed default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else
118    `torch.cuda.is_available()` ? truly-local), but the user explicitly confirms or overrides it
119    ? a GPU owner may still pick cloud, and cloud (= spend) is NEVER entered silently**. The
120    active profile is always surfaced (logged/shown). Per-profile startup checks fail/warn fast
121    (cloud-serving without GCS creds/GPU ? loud error before any job).
122    - The worker (`cmd_infer`/`cmd_train`) is already profile-agnostic (reads
123    `config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
124
125    ### 4.4 Compute placement ? and why stitch runs on Cloud Run `[design]`
126    DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In
127    cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) +
128    `wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on one metered invocation ?
129    the per-job cost ledger. Running stitch on a laptop would leave that cost
130    invisible/unattributed ? exactly what the owner wants to avoid. (Stitch stays synchronous in

```

the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + `claim\_due\_job` already supports both.)

92
93 ### 4.5 Reuse map ? most of the seams already exist `[verified]`
94 **Exists:** the 3-stage core; `\_resolve\_runner` (the single compute seam);
`StubDetectorRunner` (`replay\_csv` = byte-exact \$0 anchor); `StorageBackend` ABC + 4 backends +
`StorageRef.parse\_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control
plane (`jobs` table, `claim\_due\_job` CAS); `ExecutionPolicy WAIT\_AND\_RESUME`/`JobWaiting`
(carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip\_ai\_handoff`
(LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video
workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user\_id` columns
exist). **Net-new:** the `deployment` section; `input\_backend`; configurable output base (kill
hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-
auth header extraction; startup env detection.

95
96 ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
97 Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip
Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline
additionally:

98 1. **Captures** the source upload + the Gemini reel + the rally output
(intervals/report) into the per-video workspace (`workspaces/<video\_id>/`), instead of the no-
consent auto-delete path.
99 2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw
video kept only for the training pool, not the no-consent path).
100 3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini
(`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks
the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
101 4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING
rows** (human-reviewed labels via the annotation flow ? `data\_pipeline/`), growing the corpus ?
the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.
102 This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the
PERSONALIZATION contribution model): every consented free reel makes the owned model better.
Reuses `data\_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-
retention gated by the D10 consent.

103
104 ## 6. Honest limits ? what this does NOT do / prove
105 - A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity**
for \$0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-
side, real-video).
106 - **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a
CSV-byte claim.
107 - The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with
Gemini handoff ON until the owned model clears a floor (open Q).
108 - This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent**
gate ? both remain owner/counsel-gated before public, non-owner serving.

109
110 ## 7. Deliverables & progress tracker ? **source of truth**
111 Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off
`origin/master`, no stacking. Default-OFF where it touches core callers.

112
113 | ID | Deliverable | Depends | Gated? | Status | PR |
114 |----|-----|-----|-----|-----|----|
115 | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? |
? APPROVED 2026-06-21 |
116 | M1 | `deployment.profile` + `compute\_target` config + preset bundles +
precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? |
[#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
117 | M2 | `input\_backend`/`input\_root`; wire main CLI/API input through
`StorageRef.parse\_uri` (local=identity); storage-fake tests | M1 | safe | ? |
[#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |
118 | M3 | Configurable output/scratch base ? kill hardcoded `output/`
(`trajectory\_hybrid:237,313`, `yolo\_hybrid:407`, `gemini`); **DEFAULT stays `output/`** ; golden
+ stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? |
[#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
119 | M4 | `DeviceContext` + WASB **CPU-fallback** ; unified healthcheck/device wiring | M1 |

safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283) |

120 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? |

[#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286) (preset+startup-checks+L4 parity ?; runlog ? owner GPU) |

121 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287) (shim+image+mocked ?; real build/run = M7 ? owner) |

122 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |

123 | M8 | Per-job **cost ledger** (v6 migration: `owner\_id, gpu\_active\_seconds, wall\_seconds, bytes\_out, cost\_usd, cost\_breakdown\_json`) + **pricebook** + aggregation + `api/.../cost` | M7 | ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288) (ledger+pricebook+cost ? default-OFF; real prices + usage-emission ? owner/follow-up) |

124 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user\_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? |

[#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; consent cols + ToS/DPA ? owner/counsel) |

125 | M10 | `byo\_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |

126 | **M11** | **Data-flywheel harness (D12)**: on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data\_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |

127 | **M12** | **gdrive-api** (OAuth) StorageBackend (D14 Northstar): cloud reads the source from + writes the stitched reel **back** to the user's Google Drive next to the source (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |

128

129 **Testing strategy is a first-class deliverable** ?

[`TESTING\_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ?

[`runlogs/`](runlogs/).

130

131 **## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)***

132 **?? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**

133 1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).

134 3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).

135 4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).

136 5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).

137 6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).

138

139 **?? The remaining smaller picks are LOCKED (owner, 2026-06-21) ? see §2 D16:**

140 2. ? **Stitch placement ? co-located** in the detect job (= D4; one metered unit; the job table supports a future CPU-split non-breaking, revisit only on bursty-compile metrics).

141 7. ? **RESOLVED ? D14**: cloud needs a **gdrive-api** (OAuth) StorageBackend (read the source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.

142 8. ? **Codec ? D16**: **libx264 ENCODE** (protect parity) + **NVDEC DECODE-only** (speed, no output-byte change); NVENC-encode deferred behind a config knob.

143 9. ? **Input cap ? D16**: **hard-reject`<2GB`** for v1, as a config cap (raise to 4GB later in one line).

144 10. ? **Directory name ? `COMPUTE\_DECOUPLED\_SERVING/`** **CONFIRMED final** (already the name everywhere + in ADR-014); **CLOUD_RUN_GPU_SERVING/`** dropped.

145

146 **## 9. Definition of done**

147 - [] M1?M4 land (config + input + output-base + device layer), each green on golden

cross-OS + experiment no-regression.

148 - [] ****Profile-parity test green**** ? truly-local vs cloud-mock produce byte-identical windows + segment ranges.

149 - [] ****truly-local**** (M5) and ****cloud-serving**** (M7) each have a committed ****runlog / DEPLOY_REPORT**** with a real sample run.

150 - [] Cost ledger (M8) makes GPU + ****stitch**** + egress cost visible per job; edge auth (M9) gates multi-tenant.

151 - [] ****Data-flywheel harness (M11)**** captures?stores?shadow-evals?goldenizes consented adult jobs; the owned model is on a measured path to the D11 floor.

152 - [] ? flip Status DONE, archive to `docs/archives/past\_projects/`.

153

154 **## 10. References**

155 ****ADR lineage:**** ADR-005/008/009/010/011/012/013 ? ****ADR-014**** (`docs/archives/decisions/DECISIONS.md`). ****Strategy:**** `PLATFORM_ARCHITECTURE.md`, `HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`, `LOCAL_APPLIANCE_ARCHITECTURE.md` (Khelsutra). ****Archived predecessor:**** `docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). ****Code anchors [verified]:**** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`, `stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`, `backend/worker/__main__.py:88`, `backend/api/{uploads,job_worker}.py`, `backend/infrastructure/database.py`. ****Design provenance:**** workflow `wf_56274ff0` (4 code-mappers ? synthesis; 58 findings).

156

157 ---

158 **### Changelog**

159 - 2026-06-21 ? created from the design workflow output; M0 in progress.

160 - 2026-06-21 ? ****M0 APPROVED****; ****M1 landed**** ([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)): `deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion (D15) on `AppConfig`. Pure/additive, `profile=` default == today's behaviour (zero core change); presets touch only existing knobs (no M2 `input_backend` / M3 output-base over-reach). Suite 1068 green, cov 84.11%.

161 - 2026-06-21 ? ****M2 landed**** ([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)): `input_backend`/`input_root` + URI-aware main API/CLI input via `resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report provenance. Suite 1105 green, cov 84.84%.

162 - 2026-06-21 ? ****M3 landed**** ([#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282)): configurable output/scratch base via `backend/paths.output_base(config)` ? the 3 segmenters' chunk/handoff/temp dirs root under `output.output_dir` (default `"output"`, byte-identical; default-OFF ? no Launch Report). Fixes the latent bug where scratch ignored `--output-dir`. Suite 1114 green, cov ~84.9%.

163 - 2026-06-21 ? ****M4 landed**** ([#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283)): `DeviceContext` + native WASB ****CPU-fallback**** via a new `device="auto"` (cuda-if-available-else-cpu); explicit `"cuda"` stays strict (no silent slow-CPU downgrade), default unchanged. `"auto"` resolves before `argv` (never reaches `wasb_infer.py`); CUDA probe cached. Device-selection tested offline; real CPU-inference run owner/GPU-verified. Suite 1126 green, cov 84.96%. ****? M1?M4 (the config/input/output/device layer) COMPLETE.****

164 - 2026-06-21 ? ****M5+ batch authorized + design picks locked**** (owner): the remaining \$8 picks resolved ? ****D16**** (M6 = Cloud Run ****Jobs**** + bucket-poll; ****libx264 encode + NVDEC decode-only****; ****hard-reject <2GB**** input cap; dir name confirmed) and ****D17**** (real cloud verification budget = ****\$100/?10k cap****, delete-after). Owner gave a blanket go to write+merge the ****default-OFF code**** for every remaining gated milestone (M5,M6,M8,M9-auth,M9-consent-cols,M11-plumbing,M12), one isolated PR each, reserving the gate for the real run/spend/calibration/counsel. Grounded by gate-map workflow `wf_4849400f`.

165 - 2026-06-21 ? ****REAL M7 cloud GPU run ?**** (owner-authorized, within the D17 cap): provisioned an ****L4**** (`g2-standard-4`, `asia-southeast1-a`; Mumbai stocked-out) ? native WASB on the L4 ? ****22 rallies**** on the smoke clip ? `gs://?/outputs/05e0e577?/{intervals.csv,report.json}`; VM DELETED (\$0, ~\$0.21). ? the no-Gemini cloud run needs `--set indexing.skip_ai_handoff=true` (the fully-local path emits WASB candidate windows directly ? the long-flagged "0 rallies" cold-start fix). The reel ****stitch**** validated locally (`VideoCompiler` ? 22-segment highlights, 85 MB). `DEPLOY_REPORT` runlog = the cloud-GPU dry-run guide (`docs/CLOUD_GPU_DRYRUN_GUIDE.md`).

166 - 2026-06-21 ? **M9-auth code landed** ([#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290)): ``backend/api/auth.py`` ? Cloudflare Access **Cf-Access JWT verify** (RS256: sig+exp+iss+aud against the team JWKS; lazy ``PyJWT[crypto]``; injectable JWKS ? fully-offline tests) + ``resolve_tenant`` ? tags ``create_job(owner_id=?)``; ``SecurityConfig.{edge_auth_enabled=False,cf_team_domain,cf_access_aud}`` + CORS via ``RALLY_CORS_ALLOW_ORIGINS``. **Default-OFF**. Security review (``wf_9b4c71d0``) found NO bypass; locked alg=none + HS256-confusion rejection. **M9** ? ? the consent cols + ToS/DPA are owner/counsel; the consent migration is now v7 (M8 took v6).

167 - 2026-06-21 ? **M8 code landed** ([#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288)): per-job **cost ledger** ? ``backend/billing.py`` (pure ``usage × pricebook`` math, USD internal / INR display) + a **v6 migration** (NULLABLE cost columns on ``jobs`` + a versioned ``pricebook`` table seeded ``placeholder-v0`` at \$0; additive-NULl, idempotent) + ``record_job_cost`/`get_job_cost`/`get_video_cost`` + ``GET /api/jobs/{id}/cost`` + ``/api/videos/{id}/cost`` + ``BillingConfig`` (**default-OFF**) + a gated ``_maybe_record_job_cost`` hook. Adversarial review (``wf_498b24af``) folded a **major silent-GPU-under-bill** (an unpriced ``gpu_type`` is now surfaced, not invisible \$0) + the **honest no-op** caveat (usage emission is a follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. **M8** ? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a follow-up; placeholder ships \$0 = no user-facing change. v6 is the shared bump (M9-consent cols extend it).

168 - 2026-06-21 ? **M6 code landed** ([#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)): ``backend/compute/`` ComputeBackend registry ? ``CloudRunCompute`` dispatches a Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via ``execution_policy.poll_until`` + the storage layer (D16); ``get_compute_backend`` returns a backend ONLY for ``compute_target=cloud_run`` (else ``None`` ? the worker's in-process path, **default-OFF**). The dispatched container runs **inline** (forced ``local_gpu``+native, never re-dispatch); the worker writes a ``--done-uri`` completion marker on success. ``deploy/cloudrun/{Dockerfile,README}`` = the L4 image (CUDA+ffmpeg+fonts+``wasb`` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (``wf_87b22fa4``) folded a **blocker** (cloud ``fetch`` needs a dst ? reads silently returned ``{}`` on gcs/s3) + a poll-wait test gap + ``PolicyTimeout``?clean rc 4. Suite green, ruff+mypy clean. **M6** ? ? the real build/push/L4 run + DEPLOY_REPORT is M7 (owner, D17 budget).

169 - 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): ``backend/config/startup.py`` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (``_enforce_startup_or_exit`` ? exit 1) + the worker (``cmd_infer`/`cmd_train`` ? rc 2), **default-OFF** (probe gated behind an active profile ? the worker stays torch-free at ``profile=""``). **L4 profile-parity test** (``tests/test_profile_parity.py``): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (``wf_34671ac1``). Suite green, ruff+mypy clean. **M5** ? ? the real-GPU truly-local runlog (L5) is owner-side (template pre-staged in ``runlogs``).

170

6. assistant (2026-06-21T14:36:53.413Z)

Now let me search for the M11 tracker and look at the key files mentioned:

7. user (2026-06-21T14:36:55.002Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/boxhill17/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/boxhill17/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/gap-doc/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/gap-doc/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-
```

```

indexer/.claude/worktrees/hardchop/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/hardchop/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-
stitch/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-
stitch/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/recdoc/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/recdoc/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/resweep/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/resweep/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/tier1-windowing/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/tier1-windowing/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/w2-safeguard/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/w2-safeguard/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/experiment.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/promotion.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/utils/workspace.py

```

8. assistant (2026-06-21T14:36:56.823Z)

Perfect. Now let me read the key files from the main codebase (not worktrees):

9. user (2026-06-21T14:36:57.455Z)

```

1      """Per-video workspace path helper ? one folder per video, local?cloud identical.
2
3      See ``docs/PER_VIDEO_WORKSPACE.md``. The goal: every per-video artifact lives under
4      ONE folder keyed by the MD5 ``video_id``
5      (``backend/utils/hashing.py::compute_video_id``):
6
7      <root>/[<prefix>]/<video_id>/{source,detector,outputs,reels,logs,tmp}/...
8
9      The ``root`` is a **storage URI** (``local`` path / ``gcs://`` / ``s3://`` /
10     ``gdrive://``), so a
11     LOCAL deployment and a CLOUD (bucket) deployment share the *same relative layout* ? only
12     the root
13     differs. This module is **pure** (string/URI joins, no I/O) so it is trivially testable
14     and works
15     for every backend; the storage layer (``backend/storage``) does the actual read/write.
16
17     Two roots by design (D3): a **durable** workspace root (may be ``gcs://``) for artifacts
18     worth
19     keeping, and a **fast local compute-scratch** mirror (always local) for frame caches /
20     extracted
21     chunks that must never be written to a cloud root. ``tmp`` is the local-only bucket.
22
23     This is Phase 0: the helper + config fields only ? NOT yet wired into write sites (that
24     converges
25     in P2?P4, flag-gated by ``storage.single_folder_per_video``). No behaviour changes on
26     import.
27
28     """
29     from __future__ import annotations
30
31     import os
32     from dataclasses import dataclass
33
34

```

```

25     # Per-video subfolders. ``tmp`` is LOCAL-ONLY (frame PNGs, handoff/yolo/gemini chunks,
compile
26     # clips) ? huge + ephemeral, never pushed to a cloud workspace; cache-hygiene drops it.
27     SUBDIRS = ("source", "detector", "outputs", "reels", "logs", "tmp")
28     LOCAL_ONLY_SUBDIRS = ("tmp",)
29
30
31     def join_uri(root: str, *parts: str) -> str:
32         """Join path components onto a storage root with forward slashes.
33
34         Matches how the codebase already composes storage URIs (e.g.
35         ``f"{prefix}/weights/..."`` in
36         ``backend/worker``). Works for ``gcs://bucket``, ``s3://bucket``, ``local:///abs``
and bare
37         paths alike; empty/blank parts are skipped so ``join_uri(root, "", "x")`` ==
38         ``join_uri(root,
39         "x")``. Forward slashes are valid path separators in Python on Windows too.
40         """
41         base = root.rstrip("/")
42         # Skip empty / whitespace-only parts; for real parts strip only slashes so spaces
inside a
43         # name (e.g. a mounted "My Drive" path) are preserved.
44         tail = "/" .join(p.strip("/") for p in parts if p and p.strip())
45         return f"{base}/{tail}" if tail else base
46
47     def resolve_workspace_root(workspace_root: str, output_root: str, output_dir: str) ->
str:
48         """Durable workspace root, by precedence: ``storage.workspace_root`` ?
49         ``storage.output_root`` ? ``output.output_dir``. The first non-empty wins (non-
breaking)."""
50         return (workspace_root or "").strip() or (output_root or "").strip() or (output_dir
or "output")
51
52     @dataclass(frozen=True)
53     class VideoWorkspace:
54         """The single folder for one video. Pure path math; build with :func:`for_video`.
55
56         ``base`` = ``<root>/[<prefix>]/<video_id>``. Subfolder accessors return URIs/paths
under it.
57         """
58
59         root: str
60         video_id: str
61         prefix: str = "workspaces"
62
63         @property
64         def base(self) -> str:
65             if self.prefix:
66                 return join_uri(self.root, self.prefix, self.video_id)
67             return join_uri(self.root, self.video_id)
68
69         def path(self, sub: str = "", name: str = "") -> str:
70             """A path under this video's folder: ``base[/sub][/name]``."""
71             return join_uri(self.base, sub, name)
72
73         @property
74         def source(self) -> str:
75             return self.path("source")
76
77         @property
78         def detector(self) -> str:
79             return self.path("detector")
80

```

```

81     @property
82     def outputs(self) -> str:
83         return self.path("outputs")
84
85     @property
86     def reels(self) -> str:
87         return self.path("reels")
88
89     @property
90     def logs(self) -> str:
91         return self.path("logs")
92
93     @property
94     def tmp(self) -> str:
95         return self.path("tmp")
96
97
98     def for_video(root: str, video_id: str, prefix: str = "workspaces") -> Videoworkspace:
99         """Build the :class:`Videoworkspace` for ``video_id`` under ``root`` (a storage
100         URI)."""
101         if not video_id:
102             raise ValueError("video_id is required to build a per-video workspace")
103         return Videoworkspace(root=root, video_id=video_id, prefix=prefix)
104
105     def local_scratch_base(scratch_root: str, video_id: str) -> str:
106         """The fast LOCAL compute-scratch mirror for a video (always native FS, never a
107         cloud root).
108         Used for frame caches / extracted chunks (the ``tmp`` bucket) during compute,
109         before durable
110         artifacts are pushed to the workspace. Uses ``os.path.join`` (OS-native separators)
111         since this
112         is always a local filesystem path.
113         """
114         if not video_id:
115             raise ValueError("video_id is required for a local scratch path")
116         return os.path.join(scratch_root or os.getcwd(), video_id, "tmp")

```

10. user (2026-06-21T14:36:58.597Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23

```

```

24 Three modes (`EXPERIMENT_HARNESS.md` §5):
25 - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26   aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27   held-out loop, but variant-parameterised.
28 - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29   variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30   human spot-checks (and what two highlight reels would show side by side).
31 - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32   (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import glob
41 import json
42 import logging
43 import os
44 from dataclasses import dataclass, field
45 from datetime import datetime, timezone
46 from hashlib import sha1
47 from typing import Any, Callable, Dict, List, Optional, Sequence
48
49 from backend.eval.calibration import pct_improvement
50 from backend.eval.classifier import LogisticRegression
51 from backend.eval.gemini_refine import merge_overlaps
52 from backend.eval.metrics import Interval, match_intervals, score
53 from backend.eval.regression import gt_hash
54 from backend.eval.training import label_candidates
55 from backend.eval import eval_stats as _stats # C1: CIs + paired sign test
56 from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count
ratio
57 from backend.eval.score_calibration import fit_isotonic, fit_platt # C2: calibrate cue
scores
58
59 logger = logging.getLogger(__name__)
60
61 # Where a comparison run appends one reproducible record. Tracked location (NOT under
62 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
63 # regression.DEFAULT_BASELINE_PATH.
64 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
65 _METRICS = ("precision", "recall", "f1", "mean_iou")
66
67
68 class ExperimentError(ValueError):
69     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70     score an unlabeled video with no pinned model, or a gate referencing an absent
71     feature)."""
72
73
74 # ----- Variant
75
76
77 @dataclass
78 class Variant:
79     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
80
81     A variant turns one video's persisted candidate windows + features into rally
82     intervals. Every knob defaults to the **control** behaviour (no gate, no learned
83     filter), so a variant that merely *carries* a new, disabled cue is byte-identical
84     to control ? the core no-regression guarantee.
85
86     Fields:

```

```

87     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
88     for the leaderboard and for selecting the served path (the existing
89     `segmenter_registry` + `IndexingConfig` do the actual selection in production).
90     New cues are recorded here default-OFF.
91     - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
92     `net_crossings` feature is >= this. 0 = off. This is the eval-only gate from
93     `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
94     windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
95     - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
96     whose persisted `players_in_court` is >= this. 0 = off (no-op until the person
97     cue persists that feature).
98     - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
99     `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
100    required for `compare_unlabeled` on a learned variant.
101    - ``feature_names`` ? the persisted features the learned filter consumes. When set
102    WITHOUT `classifier_model`, `compare()` trains a fresh model per LOVO fold
103    (honest) ? the recipe, not a pinned artifact.
104    - ``threshold`` ? classifier probability cutoff. `merge_gap` ? post-filter
105    overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
106    """
107
108    name: str
109    segmenter: str = "wasb_hybrid"
110    config_overrides: Dict[str, Any] = field(default_factory=dict)
111    cue_flags: Dict[str, bool] = field(default_factory=dict)
112    min_crossings: int = 0
113    min_players: int = 0
114    classifier_model: Optional[str] = None
115    feature_names: Optional[List[str]] = None
116    threshold: float = 0.5
117    merge_gap: float = 1.0
118    # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
119    # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
120    # failure where a fixed 0.5 zeroed out a small-candidate video.
121    tune_threshold: bool = False # pick the F1-best threshold on the train fold
122    calibrate: Optional[str] = None # None | "platt" | "isotonic" ? calibrate
proba first
123
124    def is_learned(self) -> bool:
125        """True if this variant applies a learned classifier (pinned model or
recipe)."""
126        return self.classifier_model is not None or bool(self.feature_names)
127
128    def to_dict(self) -> dict:
129        return {
130            "name": self.name,
131            "segmenter": self.segmenter,
132            "config_overrides": self.config_overrides,
133            "cue_flags": self.cue_flags,
134            "min_crossings": self.min_crossings,
135            "min_players": self.min_players,
136            "classifier_model": self.classifier_model,
137            "feature_names": self.feature_names,
138            "threshold": self.threshold,
139            "merge_gap": self.merge_gap,
140            "tune_threshold": self.tune_threshold,
141            "calibrate": self.calibrate,
142        }
143
144    @classmethod
145    def from_dict(cls, d: dict) -> "Variant":
146        known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]

```

```

147         return cls(**{k: v for k, v in d.items() if k in known})
148
149     def spec_hash(self) -> str:
150         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
151         and makes a result reproducible. The pinned **control** variant always hashes
152         the
153         same, so a regression is always traceable to a recipe change, never to drift."""
154         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
155         return sha1(blob.encode()).hexdigest()[:12]
156
157     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
158     #: filter. Always the comparison reference; "rollback" = make this the served variant.
159     CONTROL = Variant(name="control")
160
161
162     # ----- persisted candidates
163
164
165     @dataclass
166     class VideoCandidates:
167         """One video's persisted detection, ready for offline re-scoring.
168
169         ``intervals`` are the candidate windows; ``features`` is column-major
170         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
171         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
172         with ``load_video_candidates``, or directly in tests.
173         """
174
175         name: str
176         intervals: List[Interval]
177         features: Dict[str, List[float]] = field(default_factory=dict)
178         gts: Optional[List[Interval]] = None
179
180
181     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
182         """True iff every candidate row carries the per-cue feature dicts the golden-feature
loaders
183         read (``shuttle`` + ``person``; ``audio`` stays optional ? it is added later by
184         ``fusion_audio.augment`` and older JSONs predate it).
185
186         The velocity-windowing / distill corpus-growth path persists a LEANER
187         ``*_golden_features.json`` row (``start``/``end`` + velocity stats, no cue dicts)
for clips
188         that never went through fusion feature extraction. The loaders that build cue-rich
189         ``VideoCandidates`` from these files ? ``ablation.load_videos``,
``fusion_compare.load_videos``
190         and ``fusion_audio.load_videos_with_audio`` ? SKIP those rather than crash on
``c["shuttle"]``,
191         so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset
cleanly.
192         An empty candidate list is not a fusion file either.
193
194         Single source of truth for the three loaders (consolidated 2026-06-18 after PRs
#209/#210
195         each landed an independent copy)."""
196         return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
197
198
199     def load_golden_videos(
200         features_dir: str,
201         *,
202         shuttle_columns: Sequence[str],
203         person_columns: Sequence[str],

```

```

204     audio_columns: Sequence[str] = (),
205     audio_policy: str = "omit",
206     skip_logger: logging.Logger = logger,
207     skip_message: str,
208     no_audio_error: str = "{path} has no audio features",
209 ) -> List["VideoCandidates"]:
210     """Shared golden-feature corpus loader behind the three structurally-identical
loaders
211     (``ablation.load_videos`` / ``fusion_compare.load_videos`` /
``fusion_audio.load_videos_with_audio``).
212
213     Reads every FULL-FUSION-SCHEMA ``*_golden_features.json`` in ``features_dir`` into a
214     ``VideoCandidates`` (intervals + the caller-supplied feature columns + golden gts).
Leaner
215     velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
216     corpus-growth path persists) are SKIPPED, not crashed on (see
``_is_fusion_schema``), so a
217     MIXED corpus loads the fusion subset cleanly.
218
219     The column lists are ALWAYS supplied by the caller ? this loader never hardcodes
columns. The
220     three callers differ ONLY in:
221
222     - ``audio_policy`` ? how the audio block is handled, checked BEFORE reading any
audio column
223     so a windowing-only file is skipped before either branch applies:
224     * ``"omit"`` ? no audio columns are loaded (``audio_columns``
ignored);
225     * ``"tolerate-missing"`` ? audio columns load, a missing block defaults them
to 0.0;
226     * ``"require"`` ? a fusion file lacking audio raises ``SystemExit``
(via
227     ``no_audio_error`` formatted with ``path``), mirroring "run ``--augment``
first".
228     - ``skip_logger`` / ``skip_message`` ? the per-caller logger + ``%s``-templated
info line
229     emitted when a windowing-only file is skipped (kept byte-identical per caller).
230     """
231     if audio_policy not in ("omit", "tolerate-missing", "require"):
232         raise ValueError(
233             f"audio_policy must be 'omit'/'tolerate-missing'/'require', got
{audio_policy!r}")
234     videos: List[VideoCandidates] = []
235     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
236         with open(path, encoding="utf-8") as f:
237             d = json.load(f)
238             cands = d["candidates"]
239             if not _is_fusion_schema(cands):
240                 skip_logger.info(skip_message, os.path.basename(path))
241                 continue
242             if audio_policy == "require" and "audio" not in cands[0]:
243                 raise SystemExit(no_audio_error.format(path=path))
244             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
245             features: Dict[str, List[float]] = {}
246             for fn in shuttle_columns:
247                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
248             for fn in person_columns:
249                 features[fn] = [float(c["person"][fn]) for c in cands]
250             if audio_policy == "tolerate-missing":
251                 for fn in audio_columns:
252                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in
cands]
253             elif audio_policy == "require":
254                 for fn in audio_columns:
255                     features[fn] = [float(c["audio"][fn]) for c in cands]

```

```

256         gts = [(float(a), float(b)) for a, b in d["gts"]]
257         videos.append(VideoCandidates(name=d["name"], intervals=intervals,
258                                     features=features, gts=gts))
259     return videos
260
261
262     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
263         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
264         `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
265         col = vc.features.get(name)
266         n = len(vc.intervals)
267         if col is None:
268             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
269                           name, vc.name)
270             return [0.0] * n
271         if len(col) != n:
272             raise ExperimentError(
273                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
274         return [float(x) for x in col]
275
276
277     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
278         cols = [_feature_column(vc, name) for name in feature_names]
279         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
280
281
282     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
283         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
284         players)."""
285         n = len(vc.intervals)
286         mask = [True] * n
287         if variant.min_crossings > 0:
288             col = _feature_column(vc, "net_crossings")
289             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
290         if variant.min_players > 0:
291             col = _feature_column(vc, "players_in_court")
292             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
293         return mask
294
295     def predict(variant: Variant, vc: VideoCandidates,
296                model: Optional[LogisticRegression] = None,
297                threshold: Optional[float] = None,
298                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
299         """Variant prediction: gate by persisted features, optionally filter by a learned
300         model, then merge. Pure and deterministic.
301
302         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
303         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
304         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
305         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
306         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
307         them.
308
309         """
310         mask = _gate_mask(variant, vc)
311         if variant.feature_names:
312             if model is None:
313                 if variant.classifier_model is None:
314                     raise ExperimentError(
315                         f"variant {variant.name!r} is learned but has no model to predict "
316                         f"with (pass a fitted model or set classifier_model)")
317                 model = LogisticRegression.load(variant.classifier_model)
318             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
319 variant.feature_names))]

```

```

317         if calibrator is not None:
318             proba = [calibrator(p) for p in proba]
319             thr = variant.threshold if threshold is None else threshold
320             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
321             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
322             return merge_overlaps(kept, gap_tol=variant.merge_gap)
323
324
325     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
326                           train_videos: Sequence[VideoCandidates], iou: float
327                           ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
328         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
329         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
330         (use the variant default). Honest: never looks at the held-out video.
331         """
332         calibrator: Optional[Callable[[float], float]] = None
333         if variant.calibrate:
334             raw: List[float] = []
335             y: List[int] = []
336             for vc in train_videos:
337                 raw.extend(float(p) for p in
338                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
339 [ ])))
339                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
340             if variant.calibrate == "platt":
341                 calibrator = fit_platt(raw, y)
342             elif variant.calibrate == "isotonic":
343                 calibrator = fit_isotonic(raw, y)
344             else:
345                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
346             threshold: Optional[float] = None
347             if variant.tune_threshold:
348                 best_t, best_f1 = variant.threshold, -1.0
349                 for k in range(1, 20):
350                     # grid 0.05..0.95 on the train
351                     t = round(0.05 * k, 2)
352                     f1s = [score(predict(variant, vc, model=model, threshold=t,
353 calibrator=calibrator),
354                               vc.gts or [], iou).f1 for vc in train_videos]
355                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
356                     if mean_f1 > best_f1:
357                         best_f1, best_t = mean_f1, t
358                     threshold = best_t
359             return calibrator, threshold
360
361 # ----- variant scoring
362
363     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
364         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
365 the
366 evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
367         X: List[List[float]] = []
368         y: List[int] = []
369         for vc in videos:
370             if vc.gts is None:
371                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
372             X.extend(_feature_matrix(vc, variant.feature_names or []))
373             y.extend(label_candidates(vc.intervals, vc.gts, iou))
374         if not X:
375             raise ExperimentError("cannot train a learned variant on zero candidates")

```

```

375         return LogisticRegression().fit(X, y, variant.feature_names)
376
377
378     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
379                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
380         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
381         mean aggregate.
382
383         Prediction policy:
384         - **static variant** (no learned filter): predict each video directly ? no
385         training, so no leakage; per-video scoring is already honest.
386         - **learned, pinned model** (`classifier_model` set): score every video with the
387         SHIPPED model. ? optimistic if those videos were in its training set ? use this
388         to report "how the shipped artifact does here", not for the promotion decision.
389         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
390         ? train on the OTHER videos, predict the held-out one. The number to trust.
391         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
392         only).
393         """
394         for vc in videos:
395             if vc.gts is None:
396                 raise ExperimentError(f"{vc.name} has no golden labels; use
397                 compare_unlabeled")
398
399         per_video: List[Dict[str, Any]] = []
400         predictions: Dict[str, List[Interval]] = {}
401
402         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
403         pinned_model = (LogisticRegression.load(variant.classifier_model)
404                         if variant.classifier_model is not None else None)
405         all_model = None
406         if recipe_lovo and not honest:
407             all_model = _train(variant, videos, iou)
408
409         for i, vc in enumerate(videos):
410             cal: Optional[Callable[[float], float]] = None
411             thr: Optional[float] = None
412             if recipe_lovo and honest:
413                 others = [v for j, v in enumerate(videos) if j != i]
414                 if not others:
415                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
416                     pinned model")
417                 model: Optional[LogisticRegression] = _train(variant, others, iou)
418                 if variant.tune_threshold or variant.calibrate: # operating point from
419                     the train fold
420                     assert model is not None # _train never returns
421                     None
422                     cal, thr = _calibrate_and_tune(variant, model, others, iou)
423             elif recipe_lovo: # honest=False ? one model over all
424                 videos
425                 model = all_model
426             else: # static variant or pinned model
427                 model = pinned_model
428                 preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
429                 assert vc.gts is not None # guarded at function top
430                 sc = score(preds, vc.gts, iou)
431                 per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
432                 for m in _METRICS}})
433             predictions[vc.name] = preds
434
435         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
436         0.0)}

```

```

429         for m in _METRICS}
430     return {
431         "variant": variant.name,
432         "spec_hash": variant.spec_hash(),
433         "is_learned": variant.is_learned(),
434         "honest_lovo": recipe_lovo and honest,
435         "per_video": per_video,
436         "aggregate": aggregate,
437         "predictions": predictions,
438     }
439
440
441     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
442         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
443         mean)."""
444         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
445         _METRICS}
446
447     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
448     Dict[str, Any]:
449         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
450         hides)."""
451         gts_by_name = {v.name: (v.gts or []) for v in videos}
452         overlaps = _segm.DEFAULT_OVERLAPS
453         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
454         ratios: List[float] = []
455         for name, preds in eval_v.get("predictions", {}).items():
456             gts = gts_by_name.get(name, [])
457             got = _segm.f1_at_overlaps(preds, gts, overlaps)
458             for ov in overlaps:
459                 f1k[ov].append(got[ov])
460             ratios.append(_segm.segment_count_ratio(preds, gts))
461
462         def _mean(xs: List[float]) -> float:
463             return sum(xs) / len(xs) if xs else 0.0
464         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
465         f1k.items()}
466         out["segment_count_ratio"] = _mean(ratios)
467         return out
468
469     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
470                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
471         """C1 + C4 honest reporting layered over a compare() pair (additive,
472         EXPERIMENT_HARNESS §6).
473
474         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
475         order),
476         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
477         when
478         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
479         """
480         a_f1 = [p["f1"] for p in eval_a["per_video"]]
481         b_f1 = [p["f1"] for p in eval_b["per_video"]]
482         return {
483             "variant_a_ci": _ci_block(eval_a),
484             "variant_b_ci": _ci_block(eval_b),
485             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
486             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
487                             "variant_b": _segmentation_block(videos, eval_b)},
488         }
489
490     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,

```

```

486         iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
487     """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
488     golden corpus and reports the **B ? A** deltas, per video and in aggregate.
489
490     The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
491     learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
492     `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
493
494     ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
495     paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
496     existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
497     shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
498     bare LOVO mean at n?6 is not trustworthy on its own.
499     """
500     if len(videos) < 1:
501         raise ExperimentError("compare needs at least one labeled video")
502     eval_a = evaluate_variant(videos, variant_a, iou, honest)
503     eval_b = evaluate_variant(videos, variant_b, iou, honest)
504
505     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
506     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
507
508     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
509     per_video_delta = [
510         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
511         for p in eval_b["per_video"]
512     ]
513
514     # One fingerprint over the whole label set ? detects "the deltas were measured
against
515     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
516     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
517
518     result: Dict[str, Any] = {
519         "iou": iou,
520         "label_set_hash": gt_hash(all_gts),
521         "num_videos": len(videos),
522         "variant_a": eval_a,
523         "variant_b": eval_b,
524         "delta": delta,
525         "delta_pct": delta_pct,
526         "per_video_delta": per_video_delta,
527     }
528     if honest_stats:
529         result["honest"] = honest_summary(videos, eval_a, eval_b)
530     return result
531
532
533     # ----- SxS on unlabeled video
534
535
536     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
537         agree_iou: float = 0.5) -> Dict[str, Any]:
538         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

```

```

539
540 With no labels there is no lift to measure ? instead this surfaces where the two
541 variants **agree vs diverge**, which is exactly what a human reviews (and what two
542 highlight reels would show side by side):
543
544 - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
545 - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
546 - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
547   the human / a later golden label adjudicates).
548
549 Both variants must be predictable without labels: a learned variant therefore needs
550 a
551 pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
552 ``metrics.match_intervals`` ? no new matching logic.
553
554 preds_a = predict(variant_a, video)
555 preds_b = predict(variant_b, video)
556 mr = match_intervals(preds_a, preds_b, agree_iou)
557
558 agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
559           for m in mr.matches]
560 agree_iou = [m.iou for m in mr.matches]
561 a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
562 b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
563
564 def _dur(ivs: List[Interval]) -> float:
565     return sum(e - s for s, e in ivs)
566
567 return {
568     "video": video.name,
569     "agree_iou": agree_iou,
570     "variant_a": variant_a.name,
571     "variant_b": variant_b.name,
572     "num_a": len(preds_a),
573     "num_b": len(preds_b),
574     "num_agreed": len(agreed),
575     "mean_agree_iou": (sum(agree_iou) / len(agree_iou)) if agree_iou else 0.0,
576     "duration_a": _dur(preds_a),
577     "duration_b": _dur(preds_b),
578     "agreed": agreed,
579     "a_only": a_only,
580     "b_only": b_only,
581 }
582
583 # ----- leaderboard / DB
584
585
586 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
587                      timestamp: Optional[str] = None) -> Dict[str, Any]:
588     """Append a compact, reproducible record of a `compare()` result to the JSON
589     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
590     `regression_baseline.json`; deliberately the size of a baseline file, not a service
591     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
592     """
593     row: Dict[str, Any] = {
594         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
595         "iou": result.get("iou"),
596         "label_set_hash": result.get("label_set_hash"),
597         "num_videos": result.get("num_videos"),
598         "variant_a": {"name": result["variant_a"]["variant"],
599                     "spec_hash": result["variant_a"]["spec_hash"],
600                     "aggregate": result["variant_a"]["aggregate"]},
601         "variant_b": {"name": result["variant_b"]["variant"],
602                     "spec_hash": result["variant_b"]["spec_hash"],

```

```

603         "aggregate": result["variant_b"]["aggregate"]},
604         "delta": result.get("delta"),
605     }
606     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
607     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
608     honest = result.get("honest") or {}
609     if honest.get("paired_delta_f1"):
610         row["honest_delta_f1"] = honest["paired_delta_f1"]
611     entries: List[Dict[str, Any]] = []
612     if os.path.isfile(path):
613         with open(path, encoding="utf-8") as f:
614             entries = json.load(f)
615     entries.append(row)
616     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
617     with open(path, "w", encoding="utf-8") as f:
618         json.dump(entries, f, indent=2)
619     logger.info("recorded experiment %s vs %s (?f1=%3f) -> %s",
620               row["variant_a"]["name"], row["variant_b"]["name"],
621               (result.get("delta") or {}).get("f1", 0.0), path)
622     return row
623
624
625     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
626                             gts: Optional[List[Interval]] = None,
627                             feature_names: Sequence[str] = ()) -> VideoCandidates:
628         """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.
629
630         ``feature_names`` selects which persisted features to lift out of each row's
`stats`
631         JSON (``duration`` is derived from the window bounds). This is the only DB-
touching
632         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
633         but works against any `Database` (including an in-memory one in tests).
634         """
635         rows = db.query_candidate_segments(video_id, detector=detector)
636         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
637         features: Dict[str, List[float]] = {}
638         for name in feature_names:
639             if name == "duration":
640                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
641             else:
642                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
643         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
644

```

11. user (2026-06-21T14:36:58.597Z)

```

1     """Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not
regress
2     at the SERVED windowing.
3
4     **Why this exists.** ``per_user_gate.should_promote_for_user`` already grades a B?A lift
5     honestly (min-N floor + ?F1 CI excludes 0 + paired sign test). But it grades whatever
scores
6     it is handed ? and the foundational bug was that those scores came from the *recall-
oriented
7     PROPOSAL* windowing (``backend.eval.windowing.PROPOSAL``), which production never
serves. So a

```

```

8      cue could clear the bar on a windowing nobody runs (Gate-A: proposal ?F1 **+0.074**)
while
9      regressing on the served one (**?0.008**). Grading harder isn't the fix; **measuring on
the
10     served windowing** is.
11
12     This module composes ? it adds no new statistics. It layers ONE rule on top of
13     ``should_promote_for_user``:
14
15     A cue is promotable only if, at the **SERVED** windowing, it WINS or at least DOES
NOT
16     REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``, and ? when its served
CI is
17     estimable ? that CI's lower bound does not sit clearly in regression territory).
18
19     A proposal-windowing win is still *reported* (it's useful signal for where to look
next), but it
20     can never carry a promotion on its own. The decision returns BOTH the served and the
proposal
21     verdicts so the eval?serve contrast is explicit and auditable.
22
23     Pure, deterministic, stdlib-only over ``eval_stats``/``per_user_gate``. Additive:
existing eval
24     paths are untouched until a caller routes its A/B through
:func:`promote_if_served_safe`.
25     """
26     from __future__ import annotations
27
28     from dataclasses import dataclass, field
29     from typing import Any, Dict, Optional, Sequence
30
31     from backend.eval.eval_stats import paired_delta_ci
32     from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
33
34     __all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]
35
36
37     def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
38                          eps: float = 0.0, n_boot: int = 2000, seed: int = 0) -> Dict[str,
Any]:
39         """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard
core).
40
41         ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED
windowing**
42         (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign
test) plus
43         a single ``regresses`` boolean:
44
45         - ``regresses=True`` when the served mean ?F1 is below ``-eps`` (a real served
drop), OR
46         the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative
band).
47         - ``regresses=False`` when the cue holds the line at the served operating point
(mean ?
48         ``-eps`` and the CI is not a confident net-negative).
49
50         ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served
mean". A tiny
51         positive ``eps`` lets noise-level wobble through (e.g. 0.005 ? half an F1 point at
n?6).
52         """
53         delta = paired_delta_ci(a_scores, b_scores, n_boot=n_boot, seed=seed)
54         mean_delta = float(delta["mean_delta"])
55         ci_low = float(delta["ci_low"])

```

```

56         ci_high = float(delta["ci_high"])
57         # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-
negative.
58         mean_regresses = mean_delta < -eps
59         ci_confidently_negative = ci_high < -eps
60         regresses = bool(mean_regresses or ci_confidently_negative)
61         return {
62             "regresses": regresses,
63             "mean_delta": mean_delta,
64             "ci_low": ci_low,
65             "ci_high": ci_high,
66             "ci_excludes_zero": bool(delta["ci_excludes_zero"]),
67             "sign_test": delta["sign_test"],
68             "n": int(delta["n"]),
69             "eps": float(eps),
70         }
71
72
73     @dataclass
74     class ServedGateResult:
75         """A promotion decision that BOTH measures the cue on the served windowing AND
reports the
76         proposal-windowing view, so the eval?serve contrast is never hidden.
77
78         - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would
promote
79         *and* the served check did not veto (the cue does not regress at the served
windowing).
80         - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not
regress served).
81         - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted
but the
82         served regression check overruled it (the exact Gate-A failure mode this module
prevents).
83         - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on
the
84         *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
85         - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs
+ sign
86         tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
87         - ``reason`` ? short human-readable explanation of the final verdict.
88         """
89
90         promote: bool
91         served_safe: bool
92         vetoed_by_served: bool
93         reason: str
94         base_decision: PromotionDecision
95         served: Dict[str, Any]
96         proposal: Optional[Dict[str, Any]] = field(default=None)
97
98         def as_dict(self) -> Dict[str, Any]:
99             return {
100                 "promote": self.promote,
101                 "served_safe": self.served_safe,
102                 "vetoed_by_served": self.vetoed_by_served,
103                 "reason": self.reason,
104                 "base_decision": self.base_decision.as_dict(),
105                 "served": self.served,
106                 "proposal": self.proposal,
107             }
108
109
110     def promote_if_served_safe(
111         served_a: Sequence[float],

```

```

112     served_b: Sequence[float],
113     *,
114     proposal_a: Optional[Sequence[float]] = None,
115     proposal_b: Optional[Sequence[float]] = None,
116     structural: bool = False,
117     min_n: int = 5,
118     p_threshold: float = 0.05,
119     served_regress_eps: float = 0.0,
120     n_boot: int = 2000,
121     seed: int = 0,
122 ) -> ServedGateResult:
123     """Promote a cue ONLY if it does not regress at the SERVED windowing (the
foundational rule).
124
125     The honest bar (`should_promote_for_user`: min-N floor + ?F1 CI excludes 0 + sign
test) is
126     applied to the **served** scores ? the operating point production runs ? so a
proposal-only
127     win can never carry a promotion. The optional `proposal_` scores are scored too
and
128     reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
their
129     own).
130
131     Args:
132         served_a/served_b: per-video held-out scores for variant A (control) and B (the
cue),
133         measured at the **SERVED** windowing. These drive the decision.
134         proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).
Reported for
135         contrast; the served regression guard still has the final say.
136         structural: forwarded to `should_promote_for_user` ? a structural guard (e.g.
Gate-A vs
137         held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
auto-*promoted* to
138         the served default if it regresses served. So for `structural=True` we report
139         `keep_on` honestly but `promote` reflects the served-safety check.
140         served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
allowed").
141
142     Returns a :class:`ServedGateResult` with the final verdict + both windowings'
evidence.
143     """
144     # 1) The honest gate, applied to the SERVED scores (never the proposal ones).
145     base = should_promote_for_user(
146         served_a, served_b,
147         structural=structural, min_n=min_n, p_threshold=p_threshold,
148         n_boot=n_boot, seed=seed,
149     )
150
151     # 2) The eval?serve guard: does B regress A at the served windowing?
152     served = served_regresses(served_a, served_b, eps=served_regress_eps,
153                             n_boot=n_boot, seed=seed)
154     served_safe = not served["regresses"]
155
156     # 3) Optional proposal-windowing view (reported, never promoting on its own).
157     proposal: Optional[Dict[str, Any]] = None
158     if proposal_a is not None and proposal_b is not None:
159         proposal = served_regresses(proposal_a, proposal_b, eps=served_regress_eps,
160                                     n_boot=n_boot, seed=seed)
161
162     # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never
add one.
163     if structural:
164         # A structural guard ships ON regardless of measured lift (base.keep_on stays

```

```

True), but
165         # it is only auto-promoted to the SERVED default when it is served-safe. If it
regresses
166         # served, promote=False (don't bake an eval-only win into production) while
keep_on holds.
167         promote = served_safe
168         vetoed = False
169         if served_safe:
170             reason = ("structural guard kept ON; served-safe (does not regress at the
served "
171                       f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for
served default")
172         else:
173             reason = ("structural guard kept ON, but NOT promoted to the served default:
it "
174                       f"REGRESSES at the served windowing (?F1
{served['mean_delta']:+.4f}); "
175                       "an eval-only win must not be served")
176         else:
177             would_promote = base.promote
178             if would_promote and not served_safe:
179                 promote = False
180                 vetoed = True
181                 reason = ("VETOED by the served-windowing guard: the cue clears the honest
bar but "
182                           f"REGRESSES at the served windowing (served ?F1
{served['mean_delta']:+.4f})"
183                           f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]). A win
on a "
184                           "windowing production doesn't serve cannot promote.")
185             elif would_promote:
186                 promote = True
187                 vetoed = False
188                 reason = ("promoted: clears the honest bar AND is served-safe (served ?F1 "
189                           f"{served['mean_delta']:+.4f}). " + base.reason)
190             else:
191                 promote = False
192                 vetoed = False
193                 reason = base.reason
194
195         return ServedGateResult(
196             promote=promote,
197             served_safe=served_safe,
198             vetoed_by_served=vetoed,
199             reason=reason,
200             base_decision=base,
201             served=served,
202             proposal=proposal,
203         )
204

```

12. assistant (2026-06-21T14:37:00.730Z)

Now let me check the database schema and migration status:

13. user (2026-06-21T14:37:01.750Z)

```

1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     from backend import billing # pure cost math (no backend imports ? no cycle)
8

```

```

9     logger = logging.getLogger(__name__)
10
11     class Database:
12         def __init__(self, db_path: str):
13             self.db_path = db_path
14             self._init_db()
15
16         @contextmanager
17         def _connect(self):
18             """Transaction scope that also CLOSES the connection.
19
20             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
21             never closes, so every call leaked its connection to GC (lingering file
22             handles on the WAL sidecars under Windows). Internal methods use this.
23             """
24             conn = self._get_connection()
25             try:
26                 with conn:
27                     yield conn
28             finally:
29                 conn.close()
30
31         def _get_connection(self):
32             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
33             # adds concurrent writers (a swallowed 'database is locked' silently drops
34             segments).
35             conn = sqlite3.connect(self.db_path, timeout=30.0)
36             conn.row_factory = sqlite3.Row
37             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
38             conn.execute("PRAGMA foreign_keys = ON")
39             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
40             conn.execute("PRAGMA busy_timeout = 30000")
41             try:
42                 conn.execute("PRAGMA journal_mode = WAL")
43             except sqlite3.OperationalError:
44                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
45             return conn
46
47         def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
48             """Run ONE bool-returning single-statement write inside a connection scope.
49
50             Captures the connect/commit/except-logger.error/return-False boilerplate shared
51             by
52             the bool-returning single-statement writers. ``error_msg`` is the caller's exact
53             per-method message (already formatted with its own ids) ? logged verbatim on a
54             swallowed write fault so each writer keeps its specific log text. Returns True
55             on
56             success, False (after logging ``error_msg``) on any exception."""
57             try:
58                 with self._connect() as conn:
59                     conn.cursor().execute(sql, params)
60                     conn.commit()
61                 return True
62             except Exception as e:
63                 logger.error(f"{error_msg}: {e}")
64                 return False
65
66         @staticmethod
67         def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
68             """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
69
70             A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
71             transaction, so an interrupted v3/v4 block can leave the column added while
72             ``user_version`` stays un-bumped ? and the next open would re-run the same
73             ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every

```

```

71         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
72     ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
73     ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
74     """
75     try:
76         cursor.execute(ddl)
77     except sqlite3.OperationalError as exc:
78         if "duplicate column name" not in str(exc).lower():
79             raise
80
81     def _init_db(self):
82         """Initializes tables for videos, play_segments, and events."""
83         with self._connect() as conn:
84             cursor = conn.cursor()
85
86             self._create_base_tables(cursor)
87
88             # Versioned migrations live here. PRAGMA user_version is the schema
89             # contract ? bump it for every change and add an ordered `if version < N`
90             # block below. Tests assert the expected version, so accidental drift fails
CI.
91             version = cursor.execute("PRAGMA user_version").fetchone()[0]
92
93             if version < 1:
94                 self._migrate_to_v1(cursor)
95
96             if version < 2:
97                 self._migrate_to_v2(cursor)
98
99             if version < 3:
100                 self._migrate_to_v3(cursor)
101
102             if version < 4:
103                 self._migrate_to_v4(cursor)
104
105             if version < 5:
106                 self._migrate_to_v5(cursor)
107
108             if version < 6:
109                 self._migrate_to_v6(cursor)
110             # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 7
111
112             conn.commit()
113
114     def _create_base_tables(self, cursor):
115         """Create the videos / play_segments / events base tables (idempotent)."""
116         # Videos Table
117         cursor.execute("""
118             CREATE TABLE IF NOT EXISTS videos (
119                 id TEXT PRIMARY KEY,
120                 filepath TEXT NOT NULL,
121                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
122                 status TEXT NOT NULL,
123                 validation_results TEXT
124             )
125         """)
126
127         # Play Segments Table (Extensible metadata JSON)
128         cursor.execute("""
129             CREATE TABLE IF NOT EXISTS play_segments (
130                 id INTEGER PRIMARY KEY AUTOINCREMENT,
131                 video_id TEXT NOT NULL,

```

```

132         start_time REAL NOT NULL,
133         end_time REAL NOT NULL,
134         duration REAL NOT NULL,
135         server TEXT,
136         receiver TEXT,
137         metadata TEXT,
138         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
139     )
140 """)
141
142 # Events Table
143 cursor.execute("""
144     CREATE TABLE IF NOT EXISTS events (
145         id INTEGER PRIMARY KEY AUTOINCREMENT,
146         segment_id INTEGER NOT NULL,
147         timestamp REAL NOT NULL,
148         type TEXT NOT NULL,
149         player TEXT,
150         details TEXT,

```

14. user (2026-06-21T14:37:02.962Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\gt_loader.py:quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)

15. assistant (2026-06-21T14:37:05.199Z)

Let me get more of the database file and search for migration details:

16. user (2026-06-21T14:37:06.562Z)

```

100         self._migrate_to_v3(cursor)
101
102     if version < 4:
103         self._migrate_to_v4(cursor)
104
105     if version < 5:
106         self._migrate_to_v5(cursor)
107
108     if version < 6:
109         self._migrate_to_v6(cursor)
110     # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 7
111
112     conn.commit()
113
114 def _create_base_tables(self, cursor):
115     """Create the videos / play_segments / events base tables (idempotent)."""
116     # Videos Table
117     cursor.execute("""
118         CREATE TABLE IF NOT EXISTS videos (
119             id TEXT PRIMARY KEY,
120             filepath TEXT NOT NULL,
121             processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
122             status TEXT NOT NULL,
123             validation_results TEXT
124         )
125     """)
126
127     # Play Segments Table (Extensible metadata JSON)
128     cursor.execute("""
129         CREATE TABLE IF NOT EXISTS play_segments (
130             id INTEGER PRIMARY KEY AUTOINCREMENT,
131             video_id TEXT NOT NULL,

```

```

132         start_time REAL NOT NULL,
133         end_time REAL NOT NULL,
134         duration REAL NOT NULL,
135         server TEXT,
136         receiver TEXT,
137         metadata TEXT,
138         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
139     )
140 """)
141
142 # Events Table
143 cursor.execute("""
144     CREATE TABLE IF NOT EXISTS events (
145         id INTEGER PRIMARY KEY AUTOINCREMENT,
146         segment_id INTEGER NOT NULL,
147         timestamp REAL NOT NULL,
148         type TEXT NOT NULL,
149         player TEXT,
150         details TEXT,
151         FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
152     )
153 """)
154
155 def _migrate_to_v1(self, cursor):
156     # v1: raw candidate detections (pre-confirmation), detector-agnostic.
157     # Common fields are columns; detector-specific metrics go in `stats` JSON,
158     # so a new segmenter reuses this table by setting its own `detector`/`stats`.
159     cursor.execute("""
160         CREATE TABLE IF NOT EXISTS candidate_segments (
161             id INTEGER PRIMARY KEY AUTOINCREMENT,
162             video_id TEXT NOT NULL,
163             detector TEXT NOT NULL,
164             chunk_index INTEGER,
165             start_time REAL NOT NULL,
166             end_time REAL NOT NULL,
167             duration REAL NOT NULL,
168             confidence REAL,
169             stats TEXT,
170             detection_params TEXT,
171             confirmed_segment_id INTEGER,
172             human_label TEXT,
173             notes TEXT,
174             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
175             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
176             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
177 DELETE SET NULL
178         )
179 """)
180 cursor.execute("""
181     CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
182     ON candidate_segments(video_id, detector)
183 """)
184 cursor.execute("PRAGMA user_version = 1")
185
186 def _migrate_to_v2(self, cursor):
187     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
188     # /api/process request. `status` is the EXECUTION state of the job
189     # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
190     # that failed validation) lives inside `result` JSON as result["status"].
191     # `result` is the full sync-era response dict (incl. report paths), so the
192     # observability report is the job's artifact, per the plan.
193     cursor.execute("""
194         CREATE TABLE IF NOT EXISTS jobs (
195             id TEXT PRIMARY KEY,
196             video_path TEXT NOT NULL,

```

```

196         video_id TEXT,
197         segmenter TEXT,
198         player_pool TEXT,
199         max_frames INTEGER,
200         status TEXT NOT NULL DEFAULT 'queued',
201         progress TEXT,
202         error TEXT,
203         result TEXT,
204         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
205         started_at TIMESTAMP,
206         finished_at TIMESTAMP
207     )
208     """
209     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
210     cursor.execute("PRAGMA user_version = 2")
211
212     def _migrate_to_v3(self, cursor):
213         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
214         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
215         # due
216         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
217         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
218         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
219         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
220         # them.
221         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
222         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
223         TIMESTAMP")
224         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
225         next_poll_at)")
226         cursor.execute("PRAGMA user_version = 3")
227
228     def _migrate_to_v4(self, cursor):
229         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
230         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
231         # path can scope rows to an owner. **NULL = local install = byte-identical to
232         # today** ? every existing row stays NULL and every existing query (which never
233         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
234         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
235         # them.
236         # Additive ONLY: no served-behavior change rides on this slice.
237         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
238         TEXT")
239         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
240         user_id TEXT")
241         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
242         # of index churn while making the eventual per-user lookups cheap.
243         cursor.execute(
244             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
245             "WHERE user_id IS NOT NULL")
246         cursor.execute(
247             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
248             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
249         cursor.execute("PRAGMA user_version = 4")
250
251     def _migrate_to_v5(self, cursor):
252         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
253         # version): the resolved per-user `fusion_config` overlay + an INLINE
254         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
255         # evidence
256         # and provenance that earned it. `is_active` pins the one row the serving bridge
257         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
258         # slice
259         # only persists/loads it ? NOTHING reads it on the served path yet.
260         #

```

```

252         # user_id            ? owner of the model (NULL allowed = the local install)
253         # version            ? monotonic per-user model version
254         # baseline_version    ? the shared baseline this was fit against (upgrade
switch)
255         # feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
256         # fusion_config       ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
257         # classifier_json     ? JSON LogisticRegression.to_dict() (None = config-only
model)
258         # promotion_evidence   ? JSON per_user_gate PromotionDecision (why it was
promoted)
259         # n_videos_at_fit      ? held-out-user corpus size when fit (min_n trust floor
input)
260         # is_active           ? the one resolved row for this user (partial-unique
below)
261         cursor.execute("""
262             CREATE TABLE IF NOT EXISTS user_models (
263                 id INTEGER PRIMARY KEY AUTOINCREMENT,
264                 user_id TEXT,
265                 version INTEGER NOT NULL DEFAULT 1,
266                 baseline_version TEXT,
267                 feature_schema_hash TEXT,
268                 fusion_config TEXT,
269                 classifier_json TEXT,
270                 promotion_evidence TEXT,
271                 n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
272                 is_active INTEGER NOT NULL DEFAULT 0,
273                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
274             )
275         """)
276         # One model version per user (NULLs compare distinct in SQLite, so the local
277         # install can still carry multiple versioned rows; the active-row guard below
278         # is what enforces a single served model).
279         cursor.execute("""
280             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
281             ON user_models(user_id, version)
282         """)
283         # At most ONE active model per user: a partial-unique index over the active
rows.
284         # (SQLite treats NULL user_id rows as distinct, so the local install's single
285         # active row is still guarded ? there is one local user.)
286         cursor.execute("""
287             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
288             ON user_models(user_id) WHERE is_active = 1
289         """)
290         cursor.execute("PRAGMA user_version = 5")
291
292     def _migrate_to_v6(self, cursor):
293         # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
294         # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
295         # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
296         # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
297         # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNS are
298         # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.
299         for col, typ in (
300             ("owner_id", "TEXT"),           # who pays (the non-owner user / tenant)
301             ("compute_backend", "TEXT"),     # local_gpu | cloud_run | ...
302             ("gpu_type", "TEXT"),            # L4 | T4 | ... (selects the per-second

```

```

GPU rate)
303         ("gpu_active_seconds", "REAL"),
304         ("wall_seconds", "REAL"),
305         ("bytes_out", "INTEGER"),           # egress
306         ("gemini_cost_usd", "REAL"),        # provider-reported, already USD
307         ("pricebook_version", "TEXT"),
308         ("cost_usd", "REAL"),               # computed total (USD = source of truth)
309         ("cost_breakdown_json", "TEXT"),    # {gpu_seconds: .., egress_gb: ..,
gemini_usd: ..}
310     ):
311         self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col}
{typ}")
312         # Partial index keeps the NULL=no-cost fast path churn-free while making per-
owner billing cheap.
313         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
314                        "WHERE owner_id IS NOT NULL")
315         # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
316         # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
317         cursor.execute("""
318             CREATE TABLE IF NOT EXISTS pricebook (
319                 version TEXT NOT NULL,
320                 resource TEXT NOT NULL,
321                 gpu_type TEXT NOT NULL DEFAULT '',
322                 unit_price_usd REAL NOT NULL DEFAULT 0.0,
323                 currency TEXT NOT NULL DEFAULT 'USD',
324                 effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
325             )
326         """)
327         cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
328                        "ON pricebook(version, resource, gpu_type)")
329         # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
(nothing
330         # user-facing changes). The owner INSERTs a real, calibrated version (real GCP
$/GPU-s,
331         # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this
seed.
332         for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
333                                    ("wall_seconds", ""), ("egress_gb", "")):
334             cursor.execute(
335                 "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type,
unit_price_usd, "
336                 "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource,
gpu_type))
337         cursor.execute("PRAGMA user_version = 6")
338
339     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
340         """Register or update a video row WITHOUT touching its child segments.
341
342         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
343         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
344         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
345         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
346         row in place; clearing segments is now an explicit, deliberate operation
347         (clear_video_data / prune_segments).
348         """
349         return self._execute_write(
350             "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
351             "ON CONFLICT(id) DO UPDATE SET "

```

```

352         "filepath=excluded.filepath, status=excluded.status, "
353         "validation_results=excluded.validation_results",
354         (video_id, filepath, status, json.dumps(validation_results)),
355         "Failed to add video",
356     )
357
358     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
359         """Return (max play_segment id, max candidate_segment id) for a video (0 if
360         none).
361         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
362         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
363         """
364         with self._connect() as conn:
365             cur = conn.cursor()
366             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
367             video_id = ?",
368                             (video_id,)).fetchone()[0]
369             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
370             video_id = ?",
371                             (video_id,)).fetchone()[0]
372             return int(p), int(c)
373
374     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
375                       play_after: Optional[int] = None, cand_upto: Optional[int] =
376                       None,
377                       cand_after: Optional[int] = None) -> None:
378         """Delete play/candidate segments by id range (events cascade from
379         play_segments).
380         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
381         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
382         rows
383         and keep the OLD ones when the run failed/produced nothing (`*_after`).
384         """
385         with self._connect() as conn:
386             cur = conn.cursor()
387             if play_upto is not None:
388                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
389                             (video_id, play_upto))
390             if play_after is not None:
391                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
392                             (video_id, play_after))
393             if cand_upto is not None:
394                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
395                 ?", (video_id, cand_upto))
396             if cand_after is not None:
397                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
398                 ?", (video_id, cand_after))
399             conn.commit()
400
401     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
402                       server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
403                       = None) -> Optional[int]:
404         try:
405             with self._connect() as conn:
406                 cursor = conn.cursor()
407                 cursor.execute(
408                     "INSERT INTO play_segments (video_id, start_time, end_time,
409                     duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
410                     (video_id, start_time, end_time, end_time - start_time, server,
411                     receiver, json.dumps(metadata or {})))
412                 conn.commit()
413                 return cursor.lastrowid

```

```

403         except Exception as e:
404             logger.error(f"Failed to add play segment: {e}")
405             return None
406
407     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
408         return self._execute_write(
409             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
410             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
411             "Failed to add event",
412         )
413
414     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
415                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
416                               stats: Optional[Dict[str, Any]] = None,
417                               detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
418         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
419         detector-specific dicts stored as JSON. Returns the new candidate id, or
420         None."""
421         try:
422             with self._connect() as conn:
423                 cursor = conn.cursor()
424                 cursor.execute(
425                     """INSERT INTO candidate_segments
426                     (video_id, detector, chunk_index, start_time, end_time, duration,
427                     confidence, stats, detection_params)
428                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
429                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
430                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
431                 )
432                 conn.commit()
433                 return cursor.lastrowid
434         except Exception as e:
435             logger.error(f"Failed to add candidate segment: {e}")
436             return None
437
438     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
439         """Record that a candidate detection was confirmed as a given play_segment."""
440         return self._execute_write(
441             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
442             (segment_id, candidate_id),
443             f"Failed to link candidate {candidate_id} to segment {segment_id}",
444         )
445
446     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
only_unlabeled: bool = False) -> List[Dict[str, Any]]:
447         """Fetch candidate detections for the annotation / fine-tuning workflow.
448         `stats` and `detection_params` are deserialized from JSON."""
449         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
450         params: List[Any] = [video_id]
451
452         if detector:
453             query += " AND detector = ?"
454             params.append(detector)
455         if only_unlabeled:
456             query += " AND human_label IS NULL"
457
458         query += " ORDER BY start_time"
459

```

```

460         candidates = []
461         with self._connect() as conn:
462             rows = conn.cursor().execute(query, params).fetchall()
463             for row in rows:
464                 d = dict(row)
465                 d["stats"] = json.loads(d["stats"] or "{}")
466                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
467                 candidates.append(d)
468         return candidates
469
470     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
471         with self._connect() as conn:
472             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
473 (video_id,)).fetchone()
474             if row:
475                 d = dict(row)
476                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
477                 return d
478             return None
479
480     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
481 List[Dict[str, Any]]:
482         """
483         Dynamically queries play segments based on filters:
484         - duration_min: float
485         - server: string
486         - receiver: string
487         - event_type: string (e.g. smash, foul)
488         """
489         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
490         params = [video_id]
491
492         if filters.get("duration_min"):
493             query += " AND r.duration >= ?"
494             params.append(filters["duration_min"])
495
496         if filters.get("server"):
497             query += " AND r.server = ?"
498             params.append(filters["server"])
499
500         if filters.get("receiver"):
501             query += " AND r.receiver = ?"
502             params.append(filters["receiver"])
503
504         if filters.get("event_type"):
505             # Check if there is an event of a specific type inside this segment
506             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
507 e.type = ?)"
508             params.append(filters["event_type"])
509
510         segments_list = []
511         with self._connect() as conn:
512             cursor = conn.cursor()
513             rows = cursor.execute(query, params).fetchall()
514
515             for row in rows:
516                 r_dict = dict(row)
517                 r_id = r_dict["id"]
518                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
519
520                 # Fetch associated events
521                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
522 (r_id,)).fetchall()
523                 r_dict["events"] = []
524                 for er in event_rows:

```

```

521         e_dict = dict(er)
522         e_dict["details"] = json.loads(e_dict["details"] or "{}")
523         r_dict["events"].append(e_dict)
524
525         segments_list.append(r_dict)
526
527     return segments_list
528
529     def clear_video_data(self, video_id: str):
530         with self._connect() as conn:
531             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
532             conn.commit()
533
534     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
535     # Writers follow the repo convention (swallow + logger.error, return bool); the
536     # worker logs loudly on a failed transition so a job can't silently wedge.
537
538     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
539                   player_pool: Optional[List[str]] = None,
540                   max_frames: Optional[int] = None,
541                   policy: Optional[Dict[str, Any]] = None,
542                   owner_id: Optional[str] = None) -> bool:
543         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
544         ExecutionPolicy
545         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
546         ``owner_id``
547         (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
548         (default)."""
549         return self._execute_write(
550             "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
551             status, policy, "
552             "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?)",
553             (job_id, video_path, segmenter,
554              json.dumps(player_pool) if player_pool is not None else None,
555              max_frames, json.dumps(policy) if policy is not None else None, owner_id),
556             f"Failed to create job {job_id}",
557         )
558
559     def mark_job_running(self, job_id: str) -> bool:
560         return self._execute_write(
561             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
562             "progress='starting' WHERE id = ?", (job_id,),
563             f"Failed to mark job {job_id} running",
564         )
565
566     def set_job_progress(self, job_id: str, progress: str) -> bool:
567         return self._execute_write(
568             "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
569             f"Failed to set progress for job {job_id}",
570         )
571
572     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
573         return self._execute_write(
574             "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
575             f"Failed to set video_id for job {job_id}",
576         )
577
578     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
579         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
580         validation) is inside `result` ? job status 'done' means 'the worker
581         finished'."""
582         return self._execute_write(
583             "UPDATE jobs SET status='done', progress='finished', result=?, "
584             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
585             (json.dumps(result), job_id),

```

```

581         f"Failed to mark job {job_id} done",
582     )
583
584     def mark_job_failed(self, job_id: str, error: str) -> bool:
585         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
586         return self._execute_write(
587             "UPDATE jobs SET status='failed', error=?, "
588             "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
589             f"Failed to mark job {job_id} failed",
590         )
591
592     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
593         """Fetch one job; `result`/`player_pool` JSON deserialized."""
594         with self._connect() as conn:
595             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
596 (job_id,)).fetchone()
597             if not row:
598                 return None
599             d = dict(row)
600             d["result"] = json.loads(d["result"]) if d["result"] else None

```

17. user (2026-06-21T14:37:07.356Z)

```

69:         ``user_version`` stays un-bumped ? and the next open would re-run the same
88:         # Versioned migrations live here. PRAGMA user_version is the schema
91:         version = cursor.execute("PRAGMA user_version").fetchone()[0]
109:         self._migrate_to_v6(cursor)
110:         # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA user_version
= 7
183:         cursor.execute("PRAGMA user_version = 1")
210:         cursor.execute("PRAGMA user_version = 2")
222:         cursor.execute("PRAGMA user_version = 3")
242:         cursor.execute("PRAGMA user_version = 4")
280:         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
290:         cursor.execute("PRAGMA user_version = 5")
292:     def _migrate_to_v6(self, cursor):
337:         cursor.execute("PRAGMA user_version = 6")

```

18. assistant (2026-06-21T14:37:09.620Z)

Now let me check for any existing flywheel or consent code in PRs and search the codebase:

19. user (2026-06-21T14:37:11.955Z)

(Bash completed with no output)

20. assistant (2026-06-21T14:37:14.185Z)

Let me check what PRs have been merged and look at the M8/M9 code that's already landed:

21. user (2026-06-21T14:37:15.395Z)

```

grep: unknown option -- p
Usage: grep [OPTION]... PATTERN [FILE]...
Try 'grep --help' for more information.

```

22. assistant (2026-06-21T14:37:17.397Z)

Let me use Grep instead:

23. user (2026-06-21T14:37:18.833Z)

```

Found 5 files
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py

```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\job_worker.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\billing.py
```

24. user (2026-06-21T14:37:19.625Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\auth.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\billing.py
```

25. assistant (2026-06-21T14:37:21.534Z)

Perfect. Now let me read the key files for M8 (billing) and M9 (auth):

26. user (2026-06-21T14:37:22.480Z)

```
1      """Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
2
3      ``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
4      (the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
**USD**
5      (matches GCP billing = one source of truth) and **displayed in INR** at a configurable
FX rate
6      (the Bangalore market). The pricebook is a versioned DB table
(``backend/infrastructure/database.py``);
7      this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates
dict, so it
8      is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
9
10     **Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every
cost is
11     ``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated
pricebook version.
12     """
13
14     from __future__ import annotations
15
16     from typing import Dict, Mapping, Tuple
17
18     # Metered resource keys ? shared by the usage dict, the pricebook rows, and the
breakdown.
19     GPU_SECONDS = "gpu_seconds"      # GPU-active seconds (the big one) ? rate is per-second,
per gpu_type
20     WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't
billed as GPU)
21     EGRESS_GB = "egress_gb"          # reel/byte egress in GB
22     GEMINI_USD = "gemini_usd"        # provider-reported cost, ALREADY in USD (a passthrough,
not rate-×)
23
24     # Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a
USD amount).
25     _RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
26
27
28     def compute_cost_usd(usage: Mapping[str, float],
29                          rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
30         """Return ``(cost_usd, breakdown)`` for one job.
31
32         ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
a resource
33         key to its USD unit price (already resolved for this pricebook version + gpu_type by
the DB
34         layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
```

```

35     Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
is ``0.0``.
36     """
37     breakdown: Dict[str, float] = {}
38     total = 0.0
39     for res in _RATE_MULTIPLIED:
40         amount = float(usage.get(res, 0.0) or 0.0)
41         rate = float(rates.get(res, 0.0) or 0.0)
42         cost = round(amount * rate, 6)      # round the LINE, then sum rounded lines so
the
43         if amount or rate:                  # breakdown reconciles to the total exactly
(no drift),
44         breakdown[res] = cost               # and record the line even at $0 for
transparency
45         total += cost
46     gemini = round(float(usage.get(GEMINI_USD, 0.0) or 0.0), 6)
47     if gemini:
48         breakdown[GEMINI_USD] = gemini
49     total += gemini
50     return round(total, 6), breakdown
51
52
53     def apply_margin(cost_usd: float, margin: float) -> float:
54         """Resale price = ``cost_usd * (1 + margin)``. ``margin`` is clamped ? 0 (never
below cost)."""
55         return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
56
57
58     def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
59         """Display conversion: USD ? INR at the configured FX rate (2 dp, the display
currency)."""
60         return round(float(cost_usd) * float(fx_inr_per_usd), 2)
61

```

27. user (2026-06-21T14:37:23.082Z)

```

1     """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id``
(M9, D9).
2
3     **DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict
NO-OP (returns
4     ``None`` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
5     ``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the
team JWKS ?
6     signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't
spooof** the
7     identity header (the explicit D9 anti-spoof requirement). Returns the verified
``user_id`` (the
8     token's ``email``, else ``sub``).
9
10    ``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it
(install the
11    ``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is
12    unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no
Cloudflare.
13    """
14
15    from __future__ import annotations
16
17    import json
18    import time
19    import urllib.request
20    from typing import Any, Callable, Mapping, Optional

```

```

21
22     # Cloudflare Access sends the signed assertion in this header (it terminates at the CF
edge).
23     CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
24     _JWKS_PATH = "/cdn-cgi/access/certs"
25     _JWKS_TTL_SEC = 600.0 # re-fetch the team JWKS at most every 10 min (keys rotate
slowly)
26
27     JwksFetcher = Callable[[str], dict]
28
29
30     class EdgeAuthError(Exception):
31         """A required Cf-Access token was missing or failed verification ? the caller
returns 401/403."""
32
33
34     # module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only;
tests inject.
35     _jwks_cache: dict[str, tuple[float, dict]] = {}
36
37
38     def _issuer(team_domain: str) -> str:
39         return team_domain.rstrip("/")
40
41
42     def _default_jwks_fetcher(team_domain: str) -> dict:
43         """Fetch the team's public JWKS (`<team_domain>/cdn-cgi/access/certs`). Network ?
only the
44         production path reaches here; tests pass an injected fetcher."""
45         url = _issuer(team_domain) + _JWKS_PATH
46         with urllib.request.urlopen(url, timeout=5) as resp: # noqa: S310 (https team
domain)
47             data: Any = json.load(resp)
48             if not isinstance(data, dict) or "keys" not in data:
49                 raise EdgeAuthError(f"unexpected JWKS shape from {url}")
50             return data
51
52
53     def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
54         """Return the JWKS for `team_domain`, cached with a TTL. An injected `fetcher`
(tests)
55         bypasses the cache so each test is deterministic."""
56         if fetcher is not None:
57             return fetcher(team_domain)
58         now = time.time()
59         cached = _jwks_cache.get(team_domain)
60         if cached and (now - cached[0]) < _JWKS_TTL_SEC:
61             return cached[1]
62         jwks = _default_jwks_fetcher(team_domain)
63         _jwks_cache[team_domain] = (now, jwks)
64         return jwks
65
66
67     def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) ->
str:
68         """Verify a Cf-Access JWT against `jwks` (RS256; checks sig + exp + issuer +
audience) and
69         return the user_id (`email` else `sub`). Raises `EdgeAuthError` on ANY
failure."""
70         try:
71             import jwt
72             from jwt import PyJWKSet
73             except Exception as exc: # pragma: no cover - only when the auth extra isn't
installed
74                 raise EdgeAuthError(

```

```

75         "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
76         "(pip install -e .[auth]) on a host with edge_auth_enabled."
77     ) from exc
78
79     try:
80         kid = jwt.get_unverified_header(token).get("kid")
81         if not kid:
82             raise EdgeAuthError("token header has no kid")
83         signing_key = PyJWKSet.from_dict(jwks)[kid]
84         claims = jwt.decode(
85             token, signing_key.key, algorithms=["RS256"],
86             audience=aud, issuer=_issuer(team_domain),
87             options={"require": ["exp", "iss", "aud"]},
88         )
89     except EdgeAuthError:
90         raise
91     except KeyError as exc: # kid not in the JWKS
92         raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
93     except Exception as exc: # jwt.* errors: bad sig / expired / wrong aud|iss /
malformed
94         raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
95
96     user = claims.get("email") or claims.get("sub")
97     if not user:
98         raise EdgeAuthError("verified token carries no email/sub claim")
99     return str(user)
100
101
102     def resolve_tenant(headers: Mapping[str, str], config: Any, *,
103                       jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
104         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
105
106         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
107         user; unchanged).
108         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
109         on any miss.
110         ``jwks_fetcher`` is injectable for offline tests.
111         """
112         sec = getattr(config, "security", None)
113         if not sec or not getattr(sec, "edge_auth_enabled", False):
114             return None # DEFAULT-OFF ? no tenant identity, byte-identical to today
115
116         team_domain = getattr(sec, "cf_team_domain", "") or ""
117         aud = getattr(sec, "cf_access_aud", "") or ""
118         if not team_domain or not aud:
119             raise EdgeAuthError(
120                 "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
121                 configured")
122
123         # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
124         support a plain
125         # dict too (tests) by trying both casings.
126         token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
127         if not token:
128             raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
129             Cloudflare Access)")
130
131         jwks = _get_jwks(team_domain, jwks_fetcher)
132         return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)

```

28. assistant (2026-06-21T14:37:25.135Z)

Now let me search for the data_pipeline directory and see what already exists for capture/storage:

29. user (2026-06-21T14:37:26.803Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/docs/data_pipeline

30. user (2026-06-21T14:37:29.063Z)

```
total 80
drwxr-xr-x 1 avidu 197609    0 Jun 21 10:18 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:54 ..
-rw-r--r-- 1 avidu 197609 4365 Jun 21 10:18 GOLDEN_DATA_SHARING.md
-rw-r--r-- 1 avidu 197609 6200 Jun 21 10:10 GOLDEN_SET_IMPLEMENTATION.md
-rw-r--r-- 1 avidu 197609 8425 Jun 21 10:10 GOLDEN_SET_PHASE1_VIDEOS.md
-rw-r--r-- 1 avidu 197609 4746 Jun 21 10:10 GOLDEN_VIDEOS.md
-rw-r--r-- 1 avidu 197609 1599 Jun 21 10:11 README.md
-rw-r--r-- 1 avidu 197609 8414 Jun 21 10:10 VIDEO_RECORDING_GUIDELINES.md
```

31. assistant (2026-06-21T14:37:30.970Z)

Let me look at the storage layer to understand the existing reuse surface:

32. user (2026-06-21T14:37:31.813Z)

```
total 52
drwxr-xr-x 1 avidu 197609    0 Jun 21 13:37 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:31 ..
-rw-r--r-- 1 avidu 197609 6326 Jun 21 13:37 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:19 __pycache__
-rw-r--r-- 1 avidu 197609 2231 Jun 20 15:34 base.py
-rw-r--r-- 1 avidu 197609 2661 Jun 20 13:48 gcs.py
-rw-r--r-- 1 avidu 197609 6910 Jun 20 15:47 gdrive.py
-rw-r--r-- 1 avidu 197609 2756 Jun 20 13:48 local.py
-rw-r--r-- 1 avidu 197609 4455 Jun 21 13:23 ref.py
-rw-r--r-- 1 avidu 197609 3728 Jun 20 13:48 s3.py
```

33. assistant (2026-06-21T14:37:33.948Z)

Now let me compile my findings. Let me also check what config sections exist:

34. user (2026-06-21T14:37:34.940Z)

```
18:class ValidationRelevanceConfig(BaseModel):
24:class ValidationConfig(BaseModel):
33:class TrackNetConfig(BaseModel):
54:class WasbIndexConfig(BaseModel):
94:class FusionConfig(BaseModel):
127:class IndexingConfig(BaseModel):
230:class OutputConfig(BaseModel):
238:class WatermarkConfig(BaseModel):
255:class StitchingConfig(BaseModel):
271:class LoggingConfig(BaseModel):
280:class SecurityConfig(BaseModel):
311:class StorageConfig(BaseModel):
348:class DeploymentConfig(BaseModel):
368:class BillingConfig(BaseModel):
387:class AppConfig(BaseModel):
```

35. user (2026-06-21T14:37:37.847Z)

```
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
```

```

286 # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287 # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288 # contain upload_dir or /api/process will reject the uploaded paths.
289 upload_dir: str = "output/uploads"
290 # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291 # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
292 max_upload_mb: int = 4096
293 max_part_mb: int = 95
294 allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296 # --- M9 edge auth (Cloudflare Access, D9) ---
297 # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298 # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299 # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300 # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301 # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302 # application's audience tag. NO secrets here ? these are public identifiers.
303 edge_auth_enabled: bool = False
304 cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305 cf_access_aud: str = "" # the CF Access application AUD tag the token must
carry
306 # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307 # backend/main.py so importing the app does no filesystem I/O; default [""]).
Credentials stay
308 # OFF ? Cf-Access is a header, not a cookie.)
309
310
311 class StorageConfig(BaseModel):
312     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313     behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0
Spaces
314     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317     (boto3/google auth chains supply credentials). See ``backend/storage/``.
318     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320     output_root: str = ""
321     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322     # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323     # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324     # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
325     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326     # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327     # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329     input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"

```

```

330     input_root: str = ""
331     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
333     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335     # the legacy flat layout is used unchanged.
336     single_folder_per_video: bool = False
337     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338     workspace_root: str = ""
339     # Cloud namespace prefix under the root so per-video folders sit tidily beside
340     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341     workspace_prefix: str = "workspaces"
342     local: dict[str, Any] = Field(default_factory=dict)
343     s3: dict[str, Any] = Field(default_factory=dict)
344     gcs: dict[str, Any] = Field(default_factory=dict)
345     gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
351
352         **Pure / additive:** the empty default profile (````) applies NO preset, so the
pipeline
353         behaves exactly as before this section existed. A profile is opt-in (``config.json``
354         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
365         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
370
371         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
372         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
373         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
374         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
375         inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
378         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the

```

```
ledger +
379         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
```

36. assistant (2026-06-21T14:37:40.157Z)

Perfect. Now let me look at what M11 should plumb in (checking if any flywheel config exists):

37. user (2026-06-21T14:37:41.537Z)

(Bash completed with no output)

38. assistant (2026-06-21T14:37:43.658Z)

Good - no flywheel config yet. Let me check the final parts - what M9 should have added for consent columns:

39. user (2026-06-21T14:37:44.463Z)

```
337         cursor.execute("PRAGMA user_version = 6")
338
339         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
340             """Register or update a video row WITHOUT touching its child segments.
341
342             Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
343             with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
344             That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
345             before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
346             row in place; clearing segments is now an explicit, deliberate operation
347             (clear_video_data / prune_segments).
348             """
349             return self._execute_write(
350                 "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
351                 "ON CONFLICT(id) DO UPDATE SET "
352                 "filepath=excluded.filepath, status=excluded.status, "
353                 "validation_results=excluded.validation_results",
354                 (video_id, filepath, status, json.dumps(validation_results)),
355                 "Failed to add video",
356             )
357
358         def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
359             """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
360
361             Captured before a (re)segmentation so the run's new rows (id > watermark) can be
362             told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
363             """
364             with self._connect() as conn:
365                 cur = conn.cursor()
366                 p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
367                                (video_id,)).fetchone()[0]
368                 c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
369                                (video_id,)).fetchone()[0]
370                 return int(p), int(c)
371
372         def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
373                           play_after: Optional[int] = None, cand_upto: Optional[int] =
```

```

None,
374             cand_after: Optional[int] = None) -> None:
375         """Delete play/candidate segments by id range (events cascade from
play_segments).
376
377         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
378         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
379         and keep the OLD ones when the run failed/produced nothing (`*_after`).
380         """
381         with self._connect() as conn:
382             cur = conn.cursor()
383             if play_upto is not None:
384                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
385             if play_after is not None:
386                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))

```

40. assistant (2026-06-21T14:37:46.553Z)

Now let me make a final check for what the README says about M11 and the consent gate:

41. user (2026-06-21T14:37:47.736Z)

```

95
96     ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
97     Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip
Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline
additionally:
98     1. **Captures** the source upload + the Gemini reel + the rally output
(intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-
consent auto-delete path.
99     2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw
video kept only for the training pool, not the no-consent path).
100    3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini
(`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks
the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
101    4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING
rows** (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ?
the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.
102    This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the
PERSONALIZATION contribution model): every consented free reel makes the owned model better.
Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-
retention gated by the D10 consent.
103
104    ## 6. Honest limits ? what this does NOT do / prove
105    - A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity**
for $0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-
side, real-video).
106    - **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a
CSV-byte claim.
107    - The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with
**Gemini handoff ON** until the owned model clears a floor (open Q).
108    - This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent**
gate ? both remain owner/counsel-gated before public, non-owner serving.
109
110    ## 7. Deliverables & progress tracker ? **source of truth**
111    Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off
`origin/master`, no stacking. Default-OFF where it touches core callers.
112
113    | ID | Deliverable | Depends | Gated? | Status | PR |
114    |----|-----|-----|-----|-----|----|
115    | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? |
**? APPROVED 2026-06-21** |

```

```

116 | M1 | `deployment.profile` + `compute_target` config + preset bundles +
precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? |
[#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
117 | M2 | `input_backend`/`input_root`; wire main CLI/API input through
`StorageRef.parse_uri` (local=identity); storage-fake tests | M1 | safe | ? |
[#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |
118 | M3 | Configurable output/scratch base ? kill hardcoded `output/`
(`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden
+ stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? |
[#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
119 | M4 | `DeviceContext` + WASB **CPU-fallback** ; unified healthcheck/device wiring | M1 |
safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/283) |
120 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed
**runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? |
[#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286) (preset+startup-
checks+L4 parity ?; runlog ? owner GPU) |
121 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) +
**containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner
(GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)
(shim+image+mocked ?; real build/run = M7 ? owner) |
122 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch
on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) |
M6 | ? owner (spend) | ? | ? |
123 | M8 | Per-job **cost ledger** (v6 migration: `owner_id, gpu_active_seconds,
wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation +
`/api/.../cost` | M7 | ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission
? owner/follow-up) |
124 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`;
DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? |
[#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290) (auth: Cf-Access JWT
verify + owner_id tag, default-OFF ?; consent cols + ToS/DPA ? owner/counsel) |
125 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit
owner approval | ? | ? |
126 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the
upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-
eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the
golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the
owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent
gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |
127 | **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the
source from + writes the stitched reel **back to the user's Google Drive next to the source**
(per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local
mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
128
129 **Testing strategy is a first-class deliverable** ?
[ `TESTING_STRATEGY.md` ](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ?
[ `runlogs/` ](runlogs/).
130
131 ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
132 **? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**
133 1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
134 3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
135 4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to
proxy).
136 5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data;
counsel-gated to M9).
137 6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until
the owned model clears the floor; capture?store?eval?goldenize meanwhile).
138
139 **? The remaining smaller picks are LOCKED (owner, 2026-06-21) ? see §2 D16:**
140 2. ? **Stitch placement ? co-located** in the detect job (= D4; one metered unit; the
job table supports a future CPU-split non-breaking, revisit only on bursty-compile metrics).
141 7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the

```

source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
142 8. ? **Codec** ? D16:**libx264 ENCODE** (protect parity) + **NVDEC DECODE-only** (speed, no output-byte change); NVENC-encode deferred behind a config knob.
143 9. ? **Input cap** ? D16:**hard-reject <2GB** for v1, as a config cap (raise to 4GB later in one line).
144 10. ? **Directory name** ? `COMPUTE_DECOUPLED_SERVING/` **CONFIRMED final** (already the name everywhere + in ADR-014); `CLOUD_RUN_GPU_SERVING/` dropped.

42. assistant (2026-06-21T14:37:49.706Z)

Perfect! Now I have all the context I need. Let me compile the comprehensive report:

43. user (2026-06-21T14:39:05.797Z)

`<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.`
You sent (first 200 of 12946 bytes): {"summary": "M11 (data-flywheel) is a mid-scope, privacy-sensitive work item that implements capture?store?shadow-eval?goldenize on consented, adult cloud-serving jobs. It reuses the eval harness (exp
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

44. assistant (2026-06-21T14:39:07.923Z)

Let me fix the JSON escaping:

45. user (2026-06-21T14:40:06.497Z)

Structured output provided successfully

46. assistant (2026-06-21T14:41:17.952Z)

Perfect. Now let me provide a final text summary with file paths and exact line references as requested:

M11 Data-Flywheel Work Item ? Precise Scope

Executive Summary

M11 (data-flywheel harness, Decision D12) is a **mid-scope, privacy-sensitive** work item that closes the loop between Gemini-on serving (D11) and owned-model training. It implements **capture ? store ? shadow-eval ? goldenize** on consented, adult cloud-serving jobs, running the owned model in parallel to Gemini to measure the gap and accumulate training data. Most machinery already exists; the gap is orchestration plumbing, consent gating, and data-pipeline integration.

What Files M11 Creates/Touches

Plan from README.md:126 (and the 4-step description at 96?102):

- Capture**: route consented uploads to per-video workspace instead of auto-delete
- Store**: durably persist in the bucket (workspace + golden-set folder)
- Shadow-eval**: dispatch owned-model predict alongside Gemini outputs
- Promote**: human-reviewed captures become golden training rows

Net-new files expected:

- `backend/flywheel.py` (main harness; capture, shadow_eval, promote_to_golden functions)
- `backend/infrastructure/database.py` ? v7 migration block for consent columns

Modified files:

- `backend/config/models.py` ? add `FlywheelConfig` class
- `backend/api/job_worker.py` or `backend/worker/__main__.py` ? plumb flywheel hooks into job

completion
- Tests: `tests/test\_flywheel.py` (new unit tests, offline/mock)

What Already Exists to Reuse

****Fully coded and tested:****

1. ****`backend/utils/workspace.for\_video`**(root: str, video_id: str, prefix: str = "workspaces") -> VideoWorkspace`** (workspace.py:98?102)

- Signature: Returns a VideoWorkspace dataclass with `.base`, `.source`, `.detector`, `.outputs`, `.reels`, `.logs`, `.tmp` properties
- Use in M11: route captured upload to `workspace.reels` for the reel, `workspace.outputs` for Gemini intervals/report

2. ****`backend/eval/experiment.compare`**(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant, iou: float = 0.5, honest: bool = True, honest_stats: bool = True) -> Dict[str, Any]`** (experiment.py:485?530)

- Performs offline A/B scoring on golden labels (or LOVO-honest per-fold training for learned variants)
- Returns: `{variant\_a, variant\_b, delta, delta\_pct, per\_video\_delta, honest: {paired\_delta\_f1, ...}}`
- Use in M11 shadow-eval: `compare(corpus, Variant.CONTROL, owned\_variant, honest=True)` to measure Gemini-vs-owned gap

3. ****`backend/eval/promotion.promote\_if\_served\_safe`**(served_a: Sequence[float], served_b: Sequence[float], *, proposal_a=None, proposal_b=None, structural=False, min_n=5, p_threshold=0.05, served_regress_eps=0.0, n_boot=2000, seed=0) -> ServedGateResult`** (promotion.py:110?122)

- Honest promotion gate: checks that variant B does not regress A at the ****served**** windowing (the eval ? serve guard)
- Returns: `ServedGateResult(promote: bool, served\_safe: bool, vetoed\_by\_served: bool, reason: str, base\_decision: PromotionDecision, served: Dict[str, Any], proposal: Optional[Dict[str, Any]])`
- Use in M11 goldenize: wrap this to decide if a video's pair (Gemini/owned) is good enough for golden labeling

4. ****Per-video workspace config**** (already wired in M2/M3)

- `StorageConfig.single\_folder\_per\_video: bool` (default False) ? enable workspace per-video folder structure
- `StorageConfig.workspace\_root: str` ? durable workspace root (empty ? falls back to output_root ? output.output_dir)
- `StorageConfig.workspace\_prefix: str = "workspaces"` ? namespace prefix
- Use: when consent_attested, set these flags at job creation time

5. ****Storage layer**** (`backend/storage/`)

- `StorageRef.parse\_uri()` (ref.py) ? parse URIs (local://, gs://, s3://)
- `StorageBackend` ABC with `fetch`, `put` methods (base.py) ? all 4 backends (local, gcs, s3, gdrive)
- Use in M11: fetch Gemini reel from job output; put captured data to workspace bucket

6. ****M8 cost ledger**** (PR #288, backend/billing.py:1?61 + database.py:292?337)

- Jobs table already has v6 columns: `owner\_id`, `compute\_backend`, `gpu\_type`, `gpu\_active\_seconds`, `wall\_seconds`, `bytes\_out`, `cost\_usd`, `pricebook\_version`, `cost\_breakdown\_json`
- All NULLABLE ? safe to build on

7. ****M9 auth plumbing**** (PR #290, backend/api/auth.py:1?127)

- `resolve\_tenant(headers: Mapping[str, str], config: Any, jwks\_fetcher=None) -> Optional[str]` ? extracts user_id from Cf-Access JWT
- `edge\_auth\_enabled` flag in `SecurityConfig` (default False)
- Jobs table already has `owner\_id` column (set by `create\_job(..., owner\_id=resolve\_tenant(...))`)

Privacy Gating Surface ? What's Needed

****What must be default-OFF:****

1. `FlywheelConfig.enabled: bool = False` ? entire flywheel is a no-op unless explicitly enabled
2. `FlywheelConfig.require\_consent\_attested: bool = False` ? if True, jobs without `consent\_attested=True` skip capture/eval
3. `FlywheelConfig.require\_adult: bool = False` ? if True, jobs without age verification skip the pipeline

****Consent fields (v7 migration, backend/infrastructure/database.py:110+):****

- `consent\_attested BOOLEAN NULL` on jobs table ? did the user explicitly consent to data use?
- `is\_adult BOOLEAN NULL` on jobs table ? was the content verified as adult (DPDP/GDPR)
- ****NOT in M11 scope (owner/counsel-gated):****
 - Age-bracket classification logic (video analysis or user input)
 - Parental consent verification token
 - DPA/ToS legal text and versioning
 - Retention policy enforcement (raw video deletion timing)

****Where consent validation happens (OWNER-DEFINED):****

- The `consent\_attested` and `is\_adult` flags are ****not auto-set by plumbing****; they come from:
 - User consent UI (at upload time or via API flag) ? owner/counsel designs this
 - Age-detection model or explicit user declaration ? owner calibrates the threshold
- The M11 plumbing ****only gates on these flags****; it does not compute them

****What is "default-OFF + a faked consent_attested" per the plan:****

- The plan says M11 plumbing runs "behind flywheel.* config = OFF + a faked consent_attested"
- Interpretation: For local dev/testing, M11 tests set `flywheel.enabled=True` + `consent\_attested=True` on mock jobs to test the pipeline without real UI
- Production: only jobs with real `consent\_attested=True` (from user consent) + `is\_adult=True` (from verification) flow into the flywheel

DB Migration Path

****Current state:****

- `PRAGMA user\_version = 6` (backend/infrastructure/database.py:337)
- M8 (PR #288) added cost-ledger columns; migration block ends at line 337

****M9 + M11 combined (v7 migration):****

...

if version < 7:

```
    self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN consent_attested BOOLEAN")
    self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN is_adult BOOLEAN")
    # future: parental_consent_token, training_pool_eligible, flywheel_eval_result as JSON
    cursor.execute("PRAGMA user_version = 7")
```

...

****Timing:****

- M9 is currently ? In progress; auth code landed (PR #290), but consent cols ****deferred to v7**** (owner/counsel-gated)
- M11 depends on M9 landing v7, OR M11 can provision v7 simultaneously if tight timeline
- ****Safest: coordinate with owner to land M9 v7 migration first, then M11 builds on it****

Test Surface ? What Can Be Tested Offline/Mocked

****100% testable without consent UI or real cloud:****

1. ****Capture routing logic:****

- Mock job with `consent\_attested=True`, verify that `backend/flywheel.capture()` routes outputs to `workspace.reels` instead of discarding
- Use in-memory `StorageRef` mock to simulate bucket I/O (no real GCS)

```

2. **Shadow-eval dispatch:**
    - Create fixture `VideoCandidates` (from experiment.py:165?178) with Gemini predictions +
      owned-model predictions
    - Call `experiment.compare(videos=[fixture], Variant.CONTROL, variant_owned)`
    - Assert the returned delta (Gemini F1 vs owned F1) and honest_stats CI

3. **Promotion gate:**
    - Use fixture scores (per-video F1 scores for both models)
    - Call `promote_if_served_safe(gemini_f1_list, owned_f1_list, min_n=3)` (low min_n for
      fixture)
    - Assert `ServedGateResult.promote` reflects whether owned beats gemini honestly (no
      proposal-only wins)

4. **Workspace routing:**
    - Mock `for_video("gs://bucket", "test-video-id")`
    - Verify `.base` = "gs://bucket/workspaces/test-video-id", `.reels` =
      "gs://bucket/workspaces/test-video-id/reels"

5. **Config validation:**
    - Load `FlywheelConfig(enabled=False)` ? flywheel ops are no-op
    - Load `FlywheelConfig(enabled=True, require_adult=True)` ? job without `is_adult` flag is
      skipped
    - Assert no side effects (stdout logs, DB writes) when disabled

**NOT testable without owner input:**
    - Actual age verification / parental consent logic
    - Real Gemini API integration (shadow-eval can mock Gemini outputs as fixture)
    - Data-pipeline annotation-workflow integration
    - Goldenize impact on model retraining (requires real training corpus + calibration)

---
```

Cleanest Isolated-PR Shape

****Single, focused PR for M11 plumbing:****

```

1. **Add FlywheelConfig to AppConfig** (backend/config/models.py, ~20 lines)
    - `enabled: bool = False`
    - `require_consent_attested: bool = False`
    - `require_adult: bool = False`
    - Optional: `eval_dual_set_path: str = ""`, `promotion_threshold_f1: float = 0.02`

2. **v7 DB migration** (backend/infrastructure/database.py, ~10 lines)
    - Add `consent_attested BOOLEAN NULL` and `is_adult BOOLEAN NULL` to jobs table
    - Bump `PRAGMA user_version = 7`

3. **backend/flywheel.py** (new, ~150 lines, all stubs)
    ```python
 def capture_if_consented(job: Dict[str, Any], config: FlywheelConfig, storage:
 StorageBackend) -> bool:
 """Route job's output to workspace if consent_attested + enabled, else return False."""
 if not config.enabled or not job.get("consent_attested"):
 return False
 if config.require_adult and not job.get("is_adult"):
 return False
 # Copy job output (Gemini reel, intervals) to workspace.reels / workspace.outputs
 workspace = for_video(config.workspace_root, job["video_id"])
 # (stub: actual copy via storage layer)
 return True

 def shadow_eval_if_promoted(job: Dict[str, Any], config: FlywheelConfig, db: Database) ->
 Optional[Dict[str, Any]]:
 """Load candidates, run experiment.compare(CONTROL=Gemini, owned), return result if
 capture succeeded."""
 if not capture_if_consented(job, config, ...):
```

```

 return None
 # Load persisted candidates (already in workspace.detector or outputs)
 candidates = load_video_candidates(db, job["video_id"], gts=None) # unlabeled
 # (stub: call experiment.compare with owned-model variant vs CONTROL)
 return {"variant_a": "control", "variant_b": "owned", "delta": {...}}

def promote_to_golden_if_safe(job: Dict[str, Any], eval_result: Dict[str, Any], config:
FlywheelConfig) -> bool:
 """Wrap promote_if_served_safe, route to data_pipeline if promote==True."""
 if not eval_result:
 return False
 served_a = eval_result.get("variant_a", {}).get("aggregate", {}).get("f1", 0.0)
 served_b = eval_result.get("variant_b", {}).get("aggregate", {}).get("f1", 0.0)
 gate = promote_if_served_safe([served_a], [served_b], min_n=1) # per-job single video
 if gate.promote:
 # (stub: call data_pipeline hook to enqueue for human annotation)
 pass
 return gate.promote
...

4. **Unit tests** (tests/test_flywheel.py, ~200 lines)
- Test disabled config ? no-op
- Test capture routing with mock workspace
- Test shadow-eval with fixture candidates
- Test promotion gate on fixture scores

5. **Wiring hook** (optional, can defer to follow-up)
- In `backend/api/job_worker.py` or worker `__main__.py`, call `promote_to_golden_if_safe()`
after job completes if flywheel enabled

Defer to follow-up PRs:
- Real consent UI validation (owner/counsel)
- Data-pipeline goldenize integration
- Dual-eval-set fixture curation
- Production observability (flywheel-eval result dashboard)

```

## Summary Table

Aspect	Status	Evidence
**Signature: workspace.for_video**	? Ready	`backend/utils/workspace.py:98?102`
**Signature: experiment.compare**	? Ready	`backend/eval/experiment.py:485?530`
**Signature: promotion.promote_if_served_safe**	? Ready	`backend/eval/promotion.py:110?122`
**DB version**	v6 (M8)	`backend/infrastructure/database.py:337` ? v7 next
**M9 auth**	? Done (auth code)	`backend/api/auth.py`, PR #290 (consent cols deferred)
**Consent schema**	? Owner/counsel-gated	Not in code; v7 migration adds flags, logic deferred
**FlywheelConfig**	Net-new	Should go in `backend/config/models.py`
**flywheel.py**	Net-new	Capture/eval/promote stubs, default-OFF
**Effort**	M (3?4 days)	Plumbing only; integration + real consent UI is L+

---

**Bottom line:** M11 is ready to plumb. The eval harness and workspace reuse are solid. The blocker is M9 v7 consent migration and owner/counsel definition of consent logic ? the engineering can ship default-OFF stubs and mock tests now, with real wiring deferred to owner/counsel input.